M-1 + ES.

DY PATIL UNIVERSITY

RamraoAdik Institute of Technology, Nerul, Navi Mumbai

Question Bank-1

Academic year: 2025-2026

Sbject: Embedded System Class: S.E. Sem : III

1. Convert $(73.301)_{10}$ into binary, octal and Hexadecimal equivalent.
2. Convert the following.
   i) $(BC)_{16} = (\ )_{10}$
   ii) $(2314)_8 = (\ )_{10}$
   iii) $(1000011)_2 = (\ )_{10}$
   iv) $(647)_{10} = (\ )_{16}$
3. Express the following numbers in decimal
   i) $(10110.0101)_2$
   ii) $(16.5)_{16}$
   iii) $(26.24)_8$
4. Give the truth table and symbol of basic logic gates.
5. Give the truth table and symbol of Universal logic gates.
6. Give the truth table and symbol of exclusive logic gates.
7. Design & implement the Half Adder using logic gates.
8. Design & implement the Full Adder using logic gates.
9. What is Multiplexer? Draw a 4:1 Multiplexer, write down the Boolean equation and design it with logic gates.
10. Explain the working of 4:1 Demultiplexer. Design it with logic gates.
11. Draw the circuit of SR Latch and explain its working with truth table.
12. Draw the circuit of D- flip flop and explain its working with truth table.
13. Draw the circuit of SR flip-flop and explain its working with truth table.

Q1 $(73.301)_{10}$ → B, O, H

concept:

647 ÷ 16 = 40.375

40 × 16 = 640

647 − 640 = 7

40 → Q

7 → R

→ (i) $(?)_B$

$(73)_{10} + (0.301)_{10}$

	Q	R
73/2	36	1
36/2	13	0
13/2	6	1
6/2	3	0
3/2	1	1
1/2	0	1

Product	
0.301 × 2 = 0.602	→ 0
0.602 × 2 = 1.204	→ 1
0.204 × 2 = 0.408	→ 0
0.408 × 2 = 0.816	→ 0
0.816 × 2 = 1.632	→ 1

$(1101.01001)_2$

→ (00 11) $(?)_O$

	Q	R
73/8	9	1
9/8	1	1
1/8	0	1

Product	
0.301 × 8 = 2.408	→ 2
0.408 × 8 = 3.264	→ 3
0.264 × 8 = 2.112	→ 2
0.112 × 8 = 0.896	→ 0

$(111.2320)_8$

(0 9 7 111) $(?)_H$

	Q	R
73/16	4	9
4/16	0	4

Product	
0.301 × 16 = 4.816	→ 4
0.816 × 16 = 13.056	→ 13
0.056 × 16 = 0.8960	→ 0
0.8960 × 16 = 14.36	→ 14

$(49.4D0E)_{16}$

(2) → (i) $(BC)_{16} \rightarrow (?)_{10}$

11 → B

C → 12

$11 \times 16^1 + 12 \times 16^0 = (188)_{10}$

→ (ii) $(2314)_8 \rightarrow (?)_{10}$

$2 \times 8^3 + 3 \times 8^2 + 1 \times 8^1 + 4 \times 8^0 = (1228)_{10}$

→ (iii) $(1000011)_2 \rightarrow (?)_{10}$

$= 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$

$= (67)_{10}$

→ (iv) $(647)_{10} \rightarrow (?)_{16}$

	Q	R
647/16	40	7
40/16	2	8
2/16	0	2

↑

$\rightarrow (287)_{16}$

Q.3 → $(10110.0101)_2 \longrightarrow (?)_{10}$

$= 1\times 2^4 + 0\times 2^3 + 1\times 2^2 + 1\times 2^1 + 0\times 2^0 + 0\times 2^{-1} + 1\times 2^{-2} + 0\times 2^{-3} + 1\times 2^{-4}$

$= (22.3125)_{10}$

→ $(16.5)_{16} \longrightarrow (?)_{10}$

$= 1\times 16^1 + 6\times 16^0 + 5\times 16^{-1}$

$= (22.3125)_{10}$

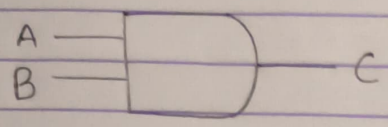
→ $(26.24)_8 \longrightarrow (?)_{10}$

$= 2\times 8^1 + 6\times 8^0 + 2\times 8^{-1} + 4\times 8^{-2}$

$= (22.3125)_{10}$

Q4 Basic Logic Gates:

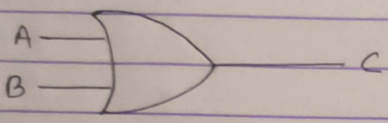
① AND GATE:



$C = A \cdot B$

A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

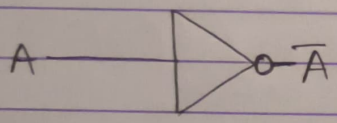
② OR GATE:



$C = A + B$

A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

③ NOT GATE:



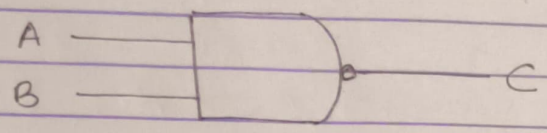
$C = \overline{A}$

A	C
0	1
1	0

Q5) Universal logic gates:

These gates are called "universal gates" because you can build all other gates using just NAND or just NOR.

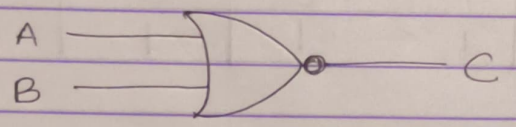
① NAND GATE:



$C = \overline{A \cdot B}$

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

② NOR GATE:

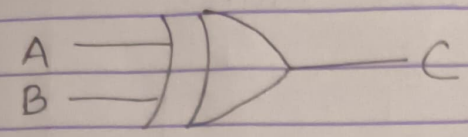


$C = \overline{A+B}$

A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

Q6) Exclusive Logic Gates:

(1) XOR GATE:

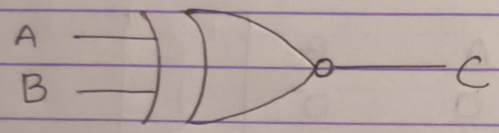


A	B	C
0	0	0
0	1	1
1	0	1
1	1	0

$$C = A \oplus B$$

$$C = \bar{A}B + A\bar{B}$$

(2) XNOR GATE:



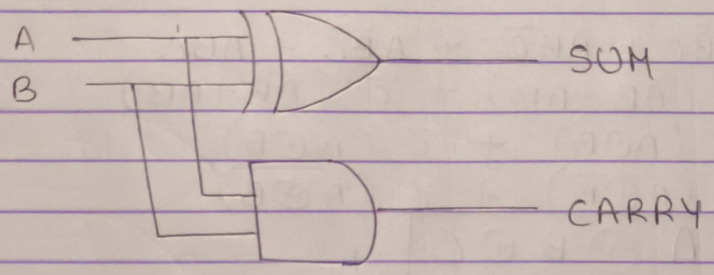
A	B	C
0	0	1
0	1	0
1	0	0
1	1	1

Q7 Half ADDER using logic gates:

A	B	Sum	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = $\bar{A}B + A\bar{B}$ (or) $A \oplus B$

CARRY = AB



Q·8 FULL ADDER using logic gates;

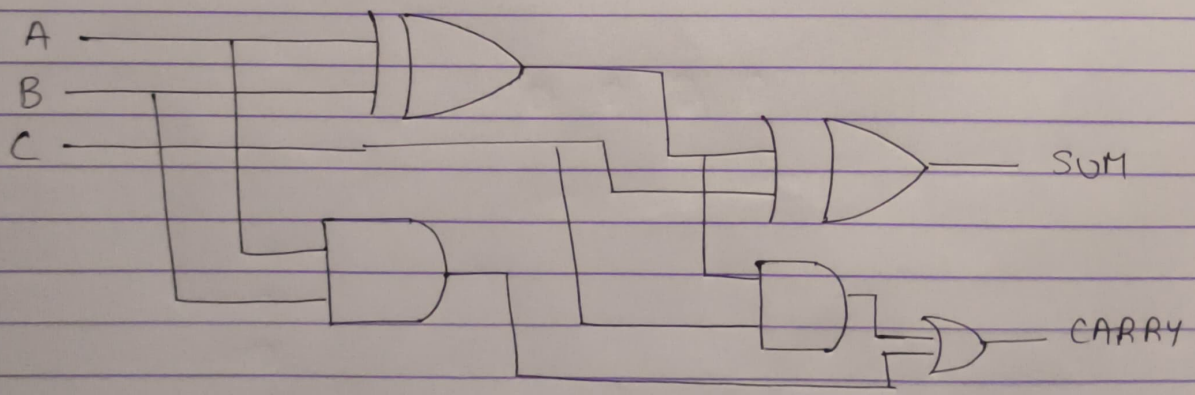
A	B	C	SUM	CARRY
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$SUM = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$
$$= C(\bar{A}B + A\bar{B}) + \bar{C}(\bar{A}B + A\bar{B})$$
$$= C(A \odot B) + \bar{C}(A \oplus B)$$
$$= C\overline{(A \oplus B)} + \bar{C}(A \oplus B)$$

$$\boxed{SUM = A \oplus B \oplus C}$$

$$CARRY = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$
$$= C(\bar{A}B + A\bar{B}) + AB(\bar{C} + C)$$

$$\boxed{CARRY = C(A \oplus B) + AB}$$



Q.9 | MUX? Draw 4:1 MUX, write down the Boolean eq. & design it with logic gates.

→ MUX is a combinational logic circuit that selects one input from multiple input signals & forwards it to a single output line.

→ It acts as a data selector - based on selection lines, it chooses which input to pass to the output.

→ A device that selects one of several input signals & transmits it to a single output.

→ i/p ⟹ 2^n

→ select lines ⟹ n

→ o/p ⟹ 1 o/p line

~~Logic Expressions: Y = ...~~

→ Applications:

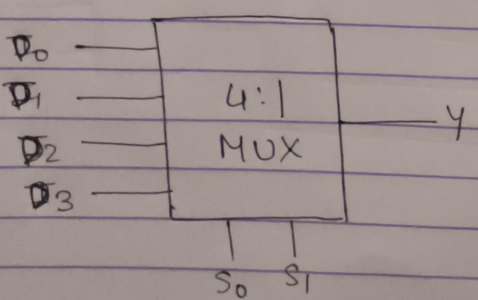
① Data routing & transmission
② Arithmetic circuits
③ Communication systems
④ function generation & logic implementation.

4:1 MUX:

4 → i/p → D_0, D_1, D_2, D_3

1 → o/p → y

2 → select lines → S_0, S_1.



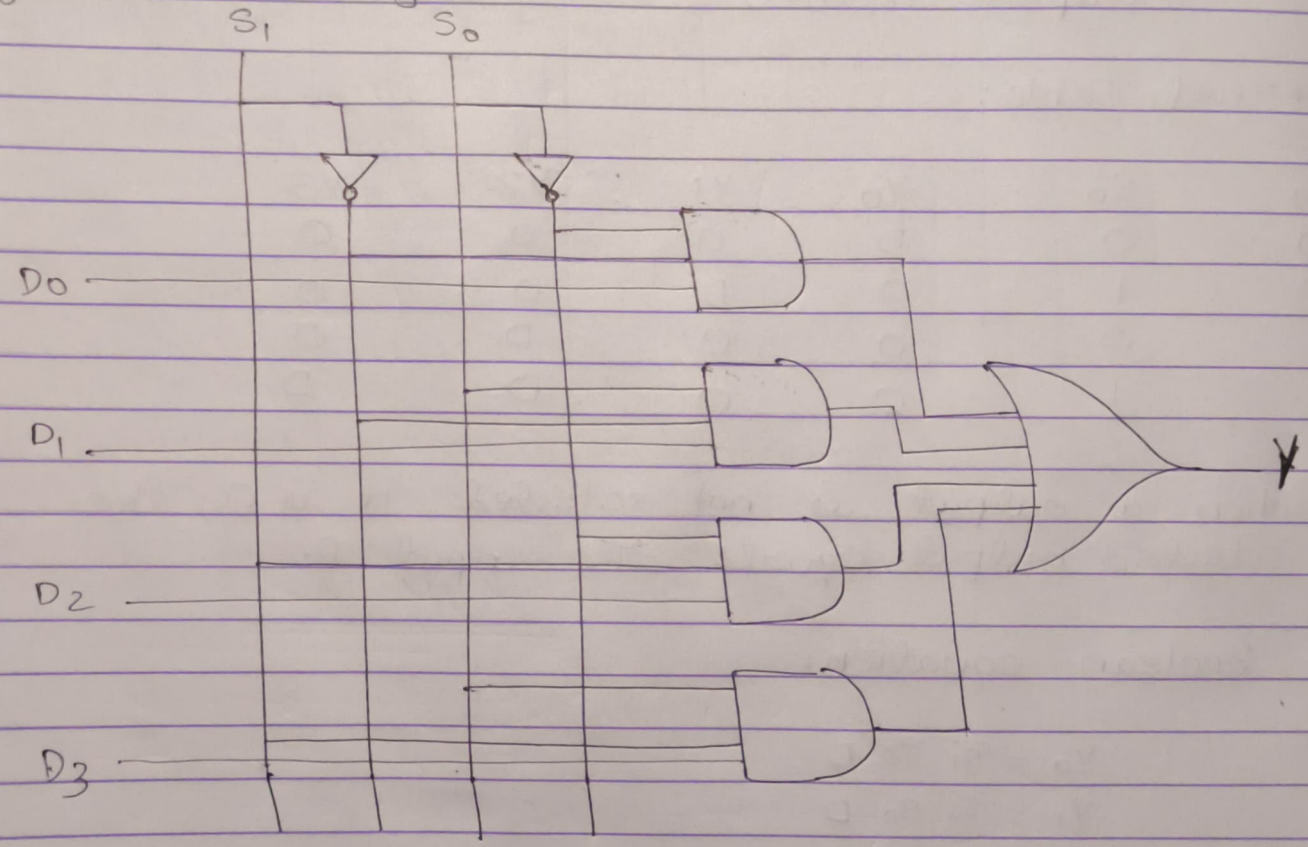
Truth Table:

S1	S0	Selected input
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

Boolean equation:

$$Y = \overline{S_1}\,\overline{S_0} D_0 + \overline{S_1} S_0 D_1 + S_1 \overline{S_0} D_2 + S_1 S_0 D_3$$

Logic Gates Design:



Q10. Explain the working of a 4:1 Demux & design it with gates (logic gates).

→ A DEMUX takes a single input & routes it to one of many outputs according to select lines. It's the opposite of MUX & a controlled distributor (one → many).

→ PARTS: ① Input : D → single data input
② Select lines : S_1 (MSB), S_0 (LSB)
③ Outputs: Y_0, Y_1, Y_2, Y_3 → only one active at a time.

→ Truth Table:

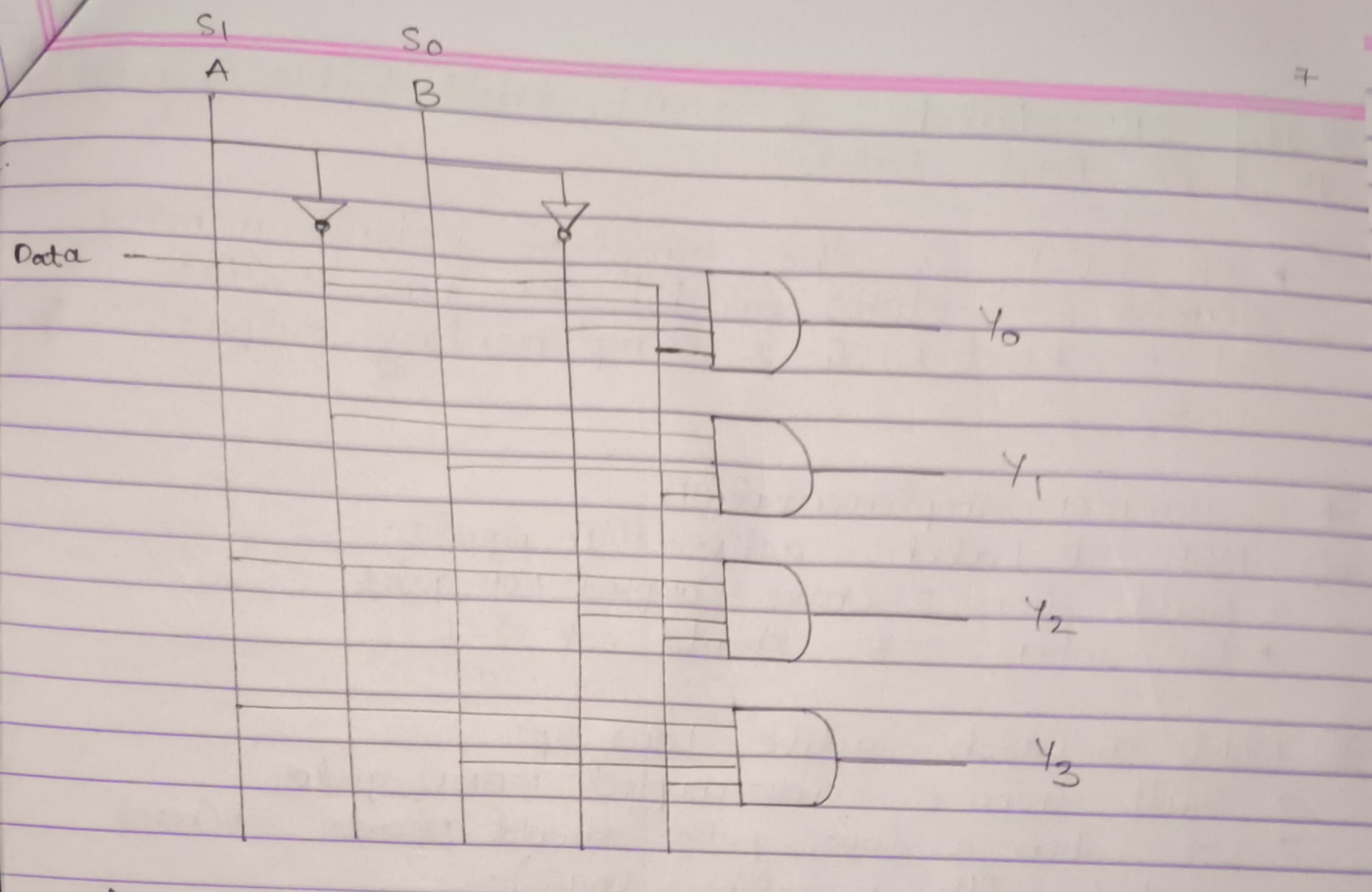
S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

when a output is not selected it is 0; the selected output equals the input D.

→ Boolean equation:

$$Y_0 = \overline{S_1}\ \overline{S_0}\ D$$
$$Y_1 = \overline{S_1}\ S_0\ D$$
$$Y_2 = S_1\ \overline{S_0}\ D$$
$$Y_3 = S_1\ S_0\ D$$

Example working;
Suppose $D=1$, $S_1=0$, $S_0=1$, → $\overline{S_1}S_0D$ is true.
$Y_1 \rightarrow 1$ & $Y_0=Y_2=Y_3=0$.
S_0 i/p 1 is routed to o/p Y_1.



Logic gates Design :

→ Each AND o/p is one of DEMUX o/ps $Y_0 - Y_3$

→ We generate complement using NOT gate $\overline{S_1}$ & $\overline{S_0}$

Q:11, 13 | SR Latch (circuit, truth table & working)
(Set-Reset Latch)

→ SR Latch is the simplest bistable memory element - it stores on bit. It has 2 i/ps (Set S, Reset R) & 2 complementary outputs Q & $\overline{Q}$

→ 2 common implementation

① NOR SR Latch: active-High inputs.
- → Built from 2 cross-coupled NOR gates
- → Use when S=1 should set Q=1

② NAND SR Latch - active-low i/p's:
- → Built from 2 - cross-coupled NAND gates
- → Use when a low pulse should assert set/reset (useful with negative logic).

→ 2 → i/p S & R
2 → o/p Q & $\overline{Q}$
(Q: current state; $\overline{Q}$: complement of Q.)

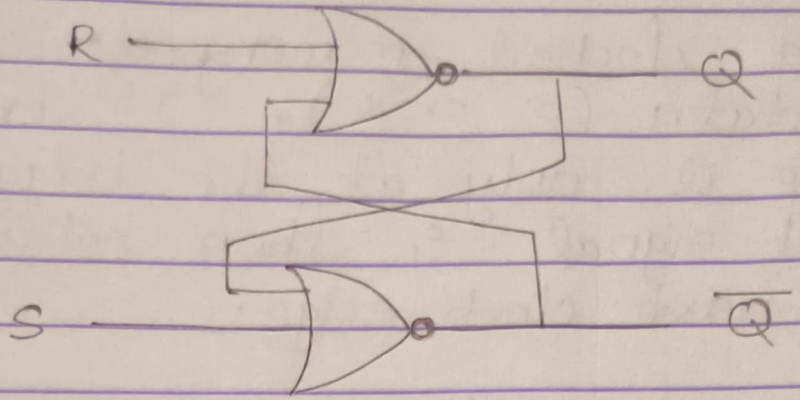
→ An SR Latch is a basic sequential circuit that stores 1-bit of information.

S	R	Q	$\overline{Q}$	Comment
0	0	N.C	N.C	No change
0	1	0	1	Reset
1	0	1	0	Set
1	1	Invalid	Invalid	Not allowed (both 1).

S=1, R=0 → Q is set to 1 (set condition).
S=0, R=1 → Q is set to 0 (Reset condition).
S=0, R=0 → Q retains its previous state (hold).
S=1, R=1 → $Q = \overline{Q}$, invalid both o/p go to 0.

→ used in F/F, RAM, control logic, etc.

Logic gates Design:



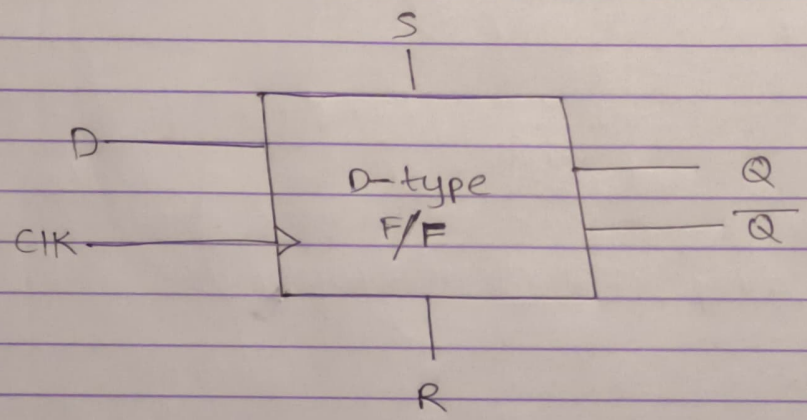
Q12. D - F/F : Data or Delay F/F

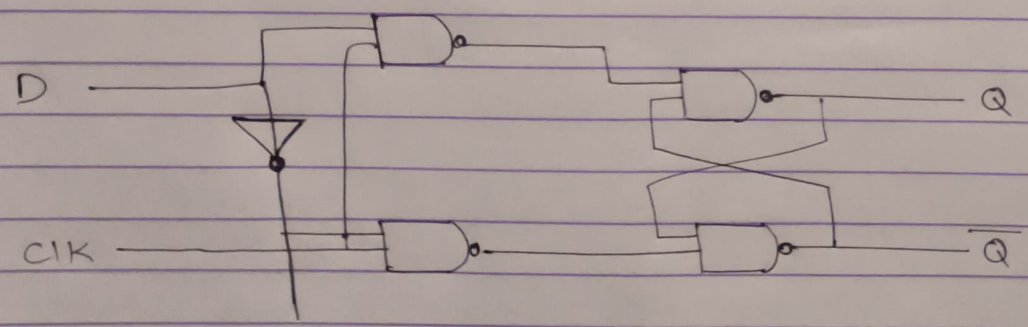
A D - F/F is a clocked memory device that stores one bit of data (0 or 1). It transfers the i/p D to o/p Q only at the triggering edge of the clock signal & then holds that value until the next clock edge.

It elements the invalid condition present in the SR F/F.

D F/F is derived from SR F/F by adding an inverter b/w S & R inputs.

$S = D$
$R = \overline{D}$
} This ensures S & R are never 1 at same time
↳ no invalid state.





NOT gate ensures invalid condition doesn't get applied.

Truth Table:

CLK	D	$Q_{(next)}$	Description.
↑	0	0	Reset
↑	1	1	Set
0 or 1	x	$Q_{(no\ change)}$	Holds previous state.

Characteristic Equation

$$Q_{next} = D$$

Working:

CLK=0 → f/F inhalds mode (no change)

CLK = 0 ⟶ 1 (rising edge):

(1) D=1 → Q =1 (set)

D=0 → Q =0 (Reset).

After that clock edge, Q remains constant until the next clock pulse.

M-2 ES.

D Y PATIL UNIVERSITY

Ramrao Adik Institute of Technology, Nerul, Navi Mumbai

Question Bank - 2

Academic year: 2025-2026

Subject: Embedded System Class: S.E. Sem : III

1. What is an embedded system? How does it differ from a general-purpose computer?
2. Explain the basic classification of Embedded Systems.
3. Describe the applications of Embedded Systems.
4. What are the different characteristics of Embedded Systems? Explain in detail.
5. What are the key components of an embedded system, and how do they interact?
6. List and explain different features of Embedded Cores.
7. Differentiate RISC and CISC architecture.
8. What is ASIC? Explain briefly.
9. Compare Microprocessors and Microcontrollers.
10. What is the Real Time Embedded System? Also explain what you mean by Hard Real time systems and Soft real time systems with suitable example.
11. What are the different memory types used in embedded systems?
12. Explain the concept of power management in embedded systems. Why is it important?

Q.1 What is an ES? What is difference b/w a ES & a General-Purpose Computer?

→ A ES is a combination of hardware & software designed to perform specific dedicated function
→ It is called es because it is embedded inside another larger system or machine to control its operation.

Example: Washing machine, microwave oven has ES that controls different tasks.

STRUCTURE:
→ A ES usually includes:
① Microcontroller / Microprocessor
② Memory (RAM, ROM)
③ Input/Output devices
④ Software (firmware) stored in ROM or Flash.

Embedded System	General Purpose Computer.
① Made for one specific task	Can perform many different tasks.
② Has simple Hardware	Has complex & flexible Hardware
③ Runs on fixed program	User can install or change program
④ Limited or no user interface (buttons, LEDs)	Full interface (keyboard, monitor, mouse).
⑤ less speed & memory	High speed & large memory
⑥ Uses less power	Needs more power
⑦ Difficult to upgrade or change	Easy to upgrade & modify
⑧ low cost & small in size	Expensive & larger in size.
⑨ Performs task automatically without user control	Needs user input for most tasks
⑩ Works continuously for long time	Works only when user turns it on.
⑪ Washing Machine, Microwave, Car Engine Control, ATM	Laptop, Desktop, Smartphone.

Q2: Basic Classification of ES?

→ ES are classified in mainly 3 ways:

1. Based on Performance & Functionality
2. Based on Real-Time Operation
3. Based on Application Type.

① Based on Performance & Functionality:

Type	Description	Example
Small-Scale	Uses a 8-bit or 16-bit microcontroller simple hardware & software	Calculator, Toys.
Medium-Scale	Uses 16/32-bit controller, may use RTOS for multitasking	Printers, Robots
Large-Scale	Uses powerful processor (32/64-bit) complex software.	Aircraft, Cars.

② Based on Real-Time Operation:

Type	Description	Example
Hard Real-Time	Must respond exactly on time	Airbag, Pacemaker
Soft Real Time	Small delay is acceptable	Music player, ATM.

③ Based on Application Type:

Type	Description	Example
Stand-Alone	Works independently	Microwave, calculator
Networked	Connected to network	Router, Smart home system
Mobile	Portable & small	Mobile phone, Camera

Q3. Applications of ES.

ES are used everywhere in daily life to make machines smart & automatic.

① Home Appliances:
→ Used to control household devices automatically
Eg: Washing Machine, Microwave oven, Air conditioner, Smart TV.

② Automobiles:
→ Used for safety, control & comfort in vehicles.
Eg: Airbag System, ECU, ABS braking, Cruise control.

③ Industrial Applications:
→ Used in factories & production systems for automation
Eg: Robots, CNC machines, Process control systems.

④ Medical Field
→ Used in life-saving & monitoring devices.
Eg: Pacemaker, ECG machine, etc.

⑤ Telecommunication:
→ Used for data transfer & communication
Eg: Routers, Mobile phones, etc.

⑥ Consumer Electronics
→ Used in entertainment & personal gadgets
Eg: Digital camera, Smartwatch, Game console.

ES are widely used in home, industry, medical, automobile, telecom & consumer products to control, monitor & automobile devices.

24. Characteristics of ES:

→ ES have special features that make them suitable for specific tasks.

① Single Functioned:

→ Performs only one specific job.

Eg: A washing machine only washes clothes.

② Tightly constrained:

→ Limited memory, power & processing speed.

→ Must work efficiently with minimum resources

③ Reactive & Real-Time:

→ Responds quickly to external inputs or events within a time limit.

Eg: Airbag system reacts instantly in an accident

④ Low Power Consumption

→ Designed to run on battery or low energy for long time.

⑤ Small size & low cost

→ Compact, simple design; cheaper to produce & maintain.

⑥ Reliable & Stable:

→ Must work correctly for years without failure.

⑦ Hardware - Software Co-design.

→ Hardware & Software are designed together for better performance.

ES are small, fast, low-power, reliable, & power one specific function efficiently.

Q5. Key components of ES & their interaction:

→ An ES mainly has 4 important parts - Hardware, Software, Memory & I/O interface.

ES → Hardware + Software + Memory + I/O all working together to perform a specific task automatically.

① Hardware:
It is physical part of system
Includes:
→ Microcontroller / Microprocessor: performs processing
→ I/p devices: sensors, switches, buttons
→ O/p devices: LEDs, displays, motors.
→ Communication ports: USB, UART, SPI, I2C.
Eg: In a fan controller, the microcontroller reads temperature & controls motor speed.

② Software (firmware):
→ The program that runs the hardware
→ Written in C or Assembly language
→ Stored permanently in ROM / Flash memory.
Eg: Code that tells washing machine when to spin or stop.

③ Memory:
→ ROM / flash: stores firmware (program)
→ RAM → stores temporary data during execution.

④ I/P & O/P Interfaces:
- I/p → sensors, switches → send signals to controller
- O/p → LEDs, motors displays → show results or perform actions.

→ How They Work Together
① I/P → sensors send signal to microcontroller
② Processing → Controller runs program
③ O/p → Sends signal to O/p device.

Eg: Temperature sensor → Microcontroller → Turns ON fan → Fan motor (O/p).

Q6: List & Explain the features of Embedded Cores:

→ It is a processor core that is used inside an ES chip
→ It performs the main processing task of the ES.

Features:

① High Performance:
→ Can handle multiple operations per clock cycle.
→ Optimized for speed & real-time performance.

② Low Power Consumption
→ Designed to work efficiently using minimum power, ideal for battery-based devices.

③ Small Size
→ Compact core fits inside chips with other components

④ Reduced Instruction Set (RISC).
→ Uses a simple set of instructions that execute fast
Eg: ARM cores use RISC architecture.

⑤ On-chip Memory & Peripherals:
→ Many cores include built-in RAM, ROM, timers, & I/O ports, reducing external hardware needs.

⑥ Real-Time Operation Support:
→ Can handle real-time tasks using ~~temp~~ interrupts & timers.

⑦ Scalability & flexibility:
→ Same core can be used for different devices by adding or removing features.

⑧ Low Cost: Integrated design reduces overall cost of the system.

Q7

RISC (Reduced Instruction Set Computer)	CISC (Computer Instruction Set Computer)
Uses small & simple set of instruction	Uses large & complex set of instruction
Each instruction executes in one clock cycle (fast).	Instructions take multiple clock cycles (slower).
Simple hardware design & control unit	Complex hardware & microprogrammed control.
Program size is larger	Program size is smaller.
Pipelining is easy to implement	Pipelining is difficult to variable instruction length
Focuses on software optimization	Focuses on hardware optimization
Uses more registers, fewer addressing modes.	Uses fewer registers, many addressing modes.
fixed instruction length	Variable instruction length.
Easier to design, test & upgrade.	More difficult & expensive to design.
Hardware cost - low development time - less	Hardware cost - high development time - more
Eg: ARM, MIPS, SPARC	Eg: AMD, Intel x86.

Q:8 ASIC:

→ Application specific Integrated Circuit.

→ It is designed to perform a specific function or task, rather than being general-purpose
→ Used in systems where a dedicated operation is needed.
→ Custom designed for a particular product or application
→ Very high speed & performance since it is optimized for one function.
→ low power consumption since only necessary circuits are included.
→ High design cost initially, but cheap for large-scale production.

Adv; High speed & efficiency
low power consumption
Compact & reliable
High performance for dedicated task

Disadv; Expensive to design & develop
Cannot be modified after manufacturing
long design time

Used in; Mobile devices, consumer electronics, automotive systems, & industrial controllers.

Example; Mobile phone chips.
Car Engine control IC.
Bitcoin mining chip.
Networking & communication chip.

Q9:

Microprocessors	Microcontroller.
It is a CPU of a computer system performs only processing	It is a complete mini computer on a single chip CPU + memory + I/O ports.
Requires external memory & I/O devices to work	Has built-in memory (RAM/ROM) & I/O ports.
Mainly used for General-purpose computing	Used for specific embedded applications.
Larger in size & needs more components.	Compact in size & self-contained.
Expensive due to extra external components	Cheaper & more cost-efficient.
Consumes more power	Consumes less power.
Slower in response because of external data transfer	Faster response - everything is inside one chip.
Used in computers, servers & laptops	Used in washing machines, cameras, cars, microwave.
Eg: Intel i3, i5, AMD	Eg: 8051, PIC, AVR, ARM Cortex.
Best suited for complex multi-task operations	Best suited for simple, dedicated control tasks.

Q10 What is Real-Time ES? Explain Types also?

(RTES):

→ It is one that responds to inputs or events within a fixed & guaranteed time limit (called deadline).

→ Used where the timing of o/p is as important as the correctness of the o/p.

→ Must complete its task on time, otherwise system failure or performance drop may occur.

Eg: Airbag controller, Pacemaker, Flight control, Video streaming, ATM.

TYPES:

① Hard RTES:

→ Task must be completed within the deadline
→ Even a small delay can cause serious failure or damage.
→ Eg: Airbag system in car, Pacemaker.

② Soft RTES:

→ Small delay is acceptable, but performance decreases.
→ NOT life-threatening or critical.
Eg: ATM machine, Music player.

Hard RTES	SOFT RTES.
→ Strict & must be met deadline	deadline is flexible, small delay is allowed.
→ May cause system crash or life risk	Only performance degradation.
→ Priority is very high	Priority is moderate

Q11: Memory Types in ES used:

ES use ROM/Flash to store programs permanently & RAM for temporary data. Types include ROM, PROM, EPROM, EEPROM, Flash, SRAM, DRAM & Cache each chosen based on speed, size & purpose.

① ROM (Read only Memory):
→ Stores permanent programs that don't change
→ Data is written once & cannot be modified easily.
Eg: TV, washing machine.

② PROM (Programmable ROM):
→ User can program it only once after manufacturing
→ Data cannot be erased or changed later.

③ EPROM (Erasable PROM):
→ Can be erased by UV light & reprogrammed.
→ Used during development & testing.

④ EEPROM (Electrically EPROM):
→ Can be erased & reprogrammed electrically, byte by byte.
→ Used for storing small data that changes like settings or calibration values.

⑤ Flash Memory:
→ Similar to EEPROM but faster & can erase large blocks at once.
→ Common in modern es for firmware updates.
Eg: Used in USB drives, mobile phones & microcontrollers

⑥ RAM (Random Access Memory):
→ Temporary storage of data during program execution
→ Data is lost when power is off
Eg: Used for variables, stack, buffers.

⑦ SRAM (Static RAM):
→ Fast & expensive, data stays as long as power is ON
→ Used for cache & small memory blocks in microcontrollers

⑧ DRAM (Dynamic RAM):
→ Cheaper & slower, needs constant refreshing.
→ Used for main memory in high end systems.

⑨ Cache Memory:
→ Very high-speed memory located inside the CPU.
→ Stores frequently used instructions to speed up processing.

⑩ Secondary Memory:
→ Used in long-term data storage

Eg: SD cards, external flash, hard drives.

Q12 Power Management in ES:

→ Power management means using different hardware & software methods to reduce power consumption in an ES without affecting performance.

→ Many ES run on battery powers.
→ Saving power increases battery life & reduces heat & energy cost.

→ To make the system energy-efficient, compact & long-lasting while maintaining good performance.

TECHNIQUES:

① Clock Gating:
Turn OFF the clock signal to unused parts of the circuit to save energy.

② Dynamic Voltage & Frequency Scaling (DVFS):
Reduce voltage & clock frequency when the processor load is low.

③ Power Gating / Sleep Mode:
Switch off unused components when not needed

④ Use of low Power Components:
Choose microcontrollers & sensors designed for low-power consumption.

⑤ Software Optimization:
Write efficient code to reduce unnecessary operations & delays.

⑥ Peripheral control:
Disable or slow down unused peripherals to save energy.

(7) Idle & Standby Modes:
Keep system in low-power modes when no activity is detected.

Benefits:
1. Longer Battery life.
2. less Heat Generation.
3. Smaller Power Supply.
4. Environment friendly.

DY PATIL UNIVERSITY — RAMRAO ADIK —

Ramrao Adik Institute of Technology, Nerul, Navi Mumbai

M-3 ES

Question Bank – 3 (Module-3)

Academic year: 2025-2025

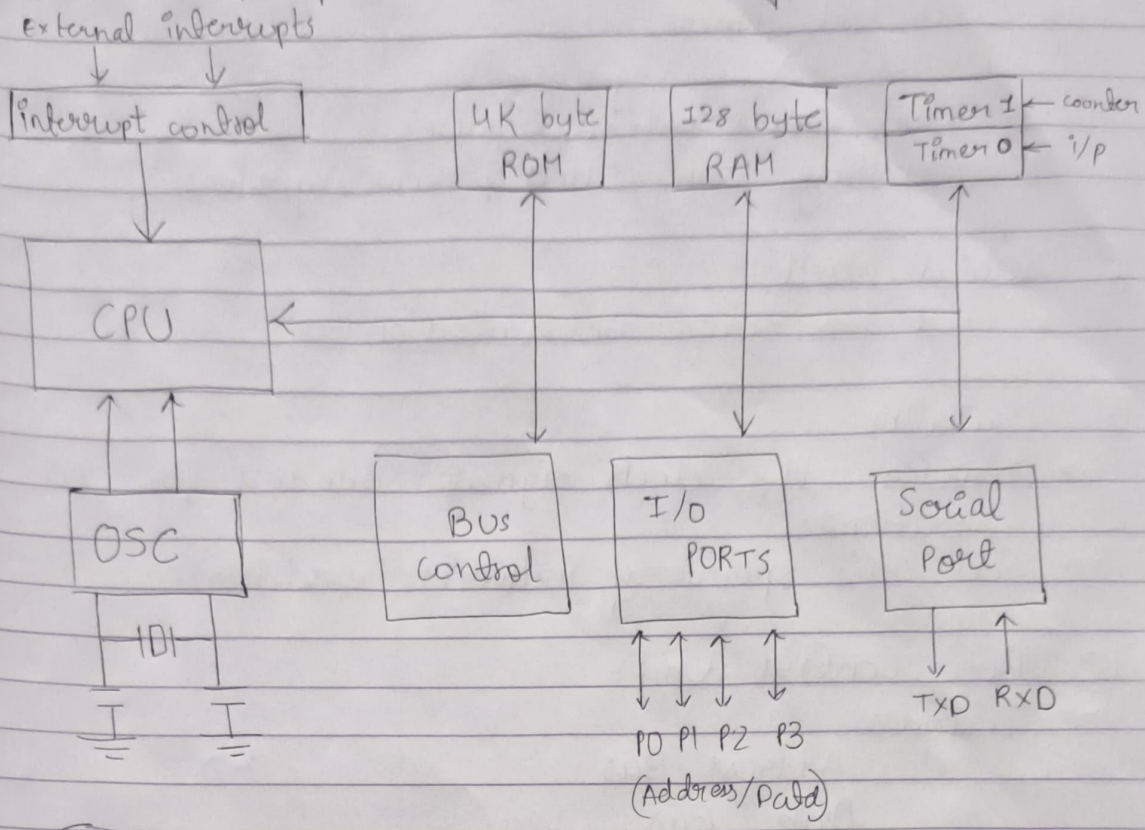
Subject: Embedded System Class: S.E. Sem : III

1. Draw the architecture of 8051 and explain in detail.
2. Draw the block diagram of 8051 and explain the main blocks.
3. Draw the pin-out of 8051 and explain the function of all pins.
4. Explain the arithmetic and logical type of instructions of 8051 with examples.
5. Explain data transfer type of instructions of 8051 with example.
6. Explain the bit manipulation type of instructions of 8051 with examples.
7. Explain branching type of instructions in 8051 with examples.
8. Explain the RET and RETI instructions of 8051 with suitable examples.
9. Explain which instruction in 8051 is used for creating a loop in program.
10. Explain all types of jump instructions in 8051 with suitable examples.
11. Write a Assembly Language Program in 8051 for Addition of two numbers.
12. Write a Assembly Language Program in 8051 for Subtraction of two numbers.
13. Write a Assembly Language Program in 8051 for Multiplication of two numbers.
14. Write a Assembly Language Program in 8051 to transfer a block of five numbers.

M-3

Q.2] Block diagram : of 8051

Q1] Draw & Explain architecture of 8051 :



(1) CPU: → Main processing unit of 8051
→ Includes ALU, Accumulator, B register, PSW, PC, SP & Registers.
→ Performs all arithmetic & logic operations.

(2) Program Memory (4KB ROM):
→ Used to store the program code
→ 4KB built inside the 8051
→ PC fetches instructions from here.

(3) Data Memory (128 Byte RAM)
→ Used to store temporary data

(4) I/O Ports
→ 4 ports × 8 bits each = 32 I/O pins
→ can be used for i/p or o/p.
→ Port 3 pins also have alternate functions.

⑤ Timers / Counters
→ 2 16-bit timers.
- used for delay generation
- Can count external pulses
→ Controlling using TMOD & TCON registers.

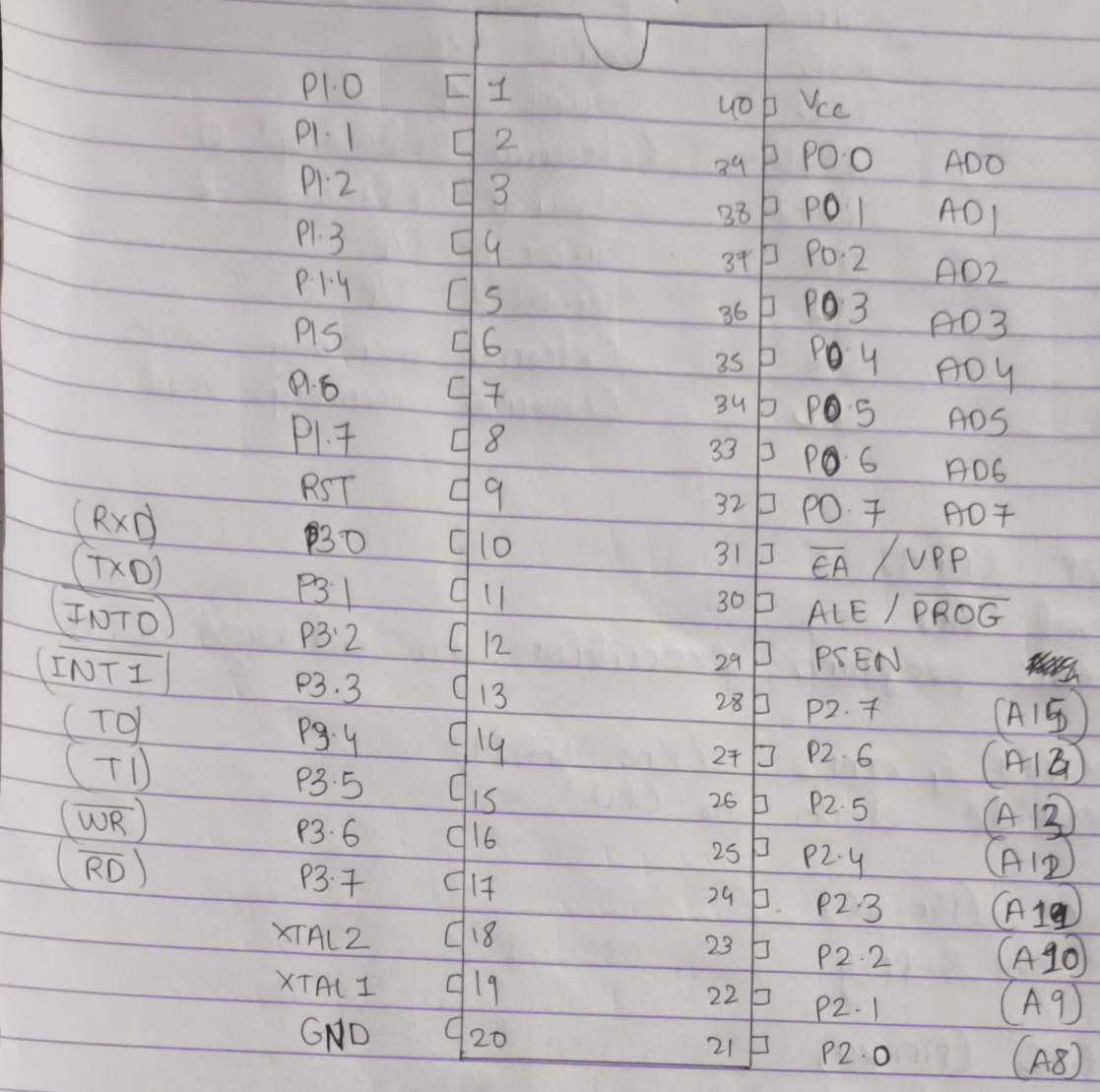
⑥ Serial Port
→ Used for serial communication

⑦ Oscillator
→ Provides the clock signal needed for CPU operations.
→ Sets the speed of instruction execution.

⑧ Bus Control Unit
→ Includes:
Address Bus
Data Bus
Control Bus
→ Purpose:
- Transfers data b/w CPU, memory, I/O & peripherals.
- Controls read/write operations.

⑨ Interrupt control.
→ Helps CPU respond immediately to urgent events.
→ Controlled using IE & IP registers.

Q3) Pin - diagram of 8051



① PORT 0 (Pin 32-39):
→ AD0 - AD7 (Address / Data bus)
→ Works as:
General I/O port or Multiplexed Address/Data bus.
→ Requires pull up resistors when used as I/O.

② PORT 1 (Pin 1-8)
→ P1.0 - P1.7
→ Pure I/O port
→ No alternate function
→ Has internal pullups.

③ PORT 2 (Pin 21-28) → P2.0 - P2.7
→ Used as: I/O port
• High-order address bus A8-A15 when accessing external memory.

④ PORT 3 (Pins (10-17))

Pin	Name	function
P3.0	RXD	Serial i/p
P3.1	TXD	Serial o/p
P3.2	INT0	External interrupt 0
P3.3	INT1	External interrupt 1
P3.4	T0	Timer 0 i/p
P3.5	T1	Timer 1 i/p
P3.6	WR	External memory write
P3.7	RD	External memory read.

⑤ RST (Pin 9)
→ Reset Pin
→ Resets program execution to address 000H.

⑥ XTAL1 & XTAL2 (Pins 18&19)
→ Provide clock for CPU

⑦ VCC (Pin 40)
→ +5v supply

⑧ GND (Pin 20)
→ Ground reference

⑨ PSEN (Pin 29)
→ Program store enable
→ Used to read external program memory

⑩ ALE (Pin 30)
→ Address Latch Enable
→ Used to demultiplex Port 0 into:
- lower address (A0-A7)
- Data bus (D0 - D7)

⑪ EA (Pin 31)
→ External Access
→ EA = 1 use internal 4KB ROM
EA = 0 use external program memory.

Q4) A & L instructions in 8051

Arithmetic Instructions:
1. ADD, ADDC → ADD
2. SUBB → subtract
3. INC/DEC → +1/-1
4. MUL AB → multiply
5. DIV AB → divide

Logical Instructions:
1. ANL → And
2. ORL → OR
3. XRL → XOR
4. CLR → Clear
5. CPL → Complement
6. RL/RR → rotate
7. RLC/RRC → rotate through carry

Arithmetic Instructions:

(1) ADD → add accumulator with byte.
syntax: ADD A, R0
meaning: A = A + R0
Eg:
```
MOV A, #25
MOV R0, #10
ADD A, R0      ; A = 25 + 10 = 35
```

(2) ADDC – add with carry
syntax: ADDC A, R1
Meaning: add carry also.
Eg:
```
ADD A, R0
ADDC A, R1     ; adds carry also.
```

(3) SUBB – subtract with Borrow
Syntax: SUBB A, R1
Example:
```
MOV A, #30
MOV R1, #10
SUBB A, R1     ; A = 30 - 10 = 20.
```

(4) INC – Increment (Add +1)
Syntax: INC A;
Eg: INC R3; R3 = R3 + 1

(5) DEC – Decrement (Subtract -1)
Syntax: DEC A;
Eg: DEC R0; R0 = R0 - 1

⑥ MUL AB - Multiply A & B (8-bit × 8-bit)

Syntax; MUL AB;

Eg: MOV A, #5
MOV B, #4
MUL AB ; A=20 (result); B=00 (overflow)

⑦ DIV AB - Divide A/B

Syntax; DIV AB;

Eg: MOV A, #20
MOV B, #5
DIV AB ; A=4 (Quotient), B=0 (Remainder).

Logical Instructions:

① ANL - Logical AND:

Sy. ; ANL A

Eg: MOV A, # OFH
ANL A, #F0H ; A= OF AND F0 =00

② ORL - logical OR

Sy: ORL A

Eg: MOV A, #0FH
ORL A, #F0H ; A=FFH

③ XRL - logical XOR

Syn: XRL A

Eg: MOV A; #55H
XRL A, #FFH ; A=AA.

④ CPL complement (NOT operation).

CPL A → invert all bits of A
CPL C → invert carry flag:

Eg: MOV A, #0FH
CPL A ; A=F0H

⑤ CLR → Clear Make Zero.

CLR A → A=00 Eg: CLR A ; A=00
CLR C → clear carry flag CLR C ; Carry =0

⑥ RL, RR – Rotate left (RL) / right (RR):

Eg: MOV A, #10010001b
RL A ; rotate left

⑦ RLC, RRC – Rotate with carry
→ used in multi-byte shift operations.

Eg: RRC A
→ Rotate right through carry → LSB goes to carry

RLC A
→ Rotate left through carry → MSB goes to carry.

Eg: MOV A, #10010010b
CLR C ; C=0
RRC A ; rotate right through carry

Eg: MOV A, #10010010b
CLR C ; C=0
RLC A ; rotate left through carry.

Q5) Explain data transfer type instructions of 8051 with examples: (5)

(1) MOV - move data.
→ used to transfer data b/w registers, memory & immediate values.

(a) Move immediate data.
Eg: MOV A, #25 ; A=25

(b) Move data b/w registers.
Eg: MOV R1, A ; R1 = A

(c) Move data b/w register & memory
Eg: MOV A, 30H ; A= data at address 30H
MOV 40H, A ; store A in RAM location 40H

(d) Move using indirect addressing:
Eg: MOV A, @R0 ; A=RAM(R0)
MOV @R1, A ; RAM(R1) = A.

(2) MOVX — Move external memory:
→ used to access external memory.

Eg: MOV DPTR, #2000H
MOVX A, @DPTR ; A= external_memory[2000H]

MOVX @DPTR, A ; external_memory (2000H) = A

(3) MOVC - moved code memory.
→ used to read lookup tables stored in program memory (ROM)
Eg: MOV DPTR, # TABLE
MOVC A, @A + DPTR ; A= code_memory (DPTR + A)

(4) PUSH → push data to stack.
→ PUSH 30H; push value of RAM(30H) onto stack

(5) POP - pop data from stack.

Eg: POP 40 ; pop into RAM (40H)

(6) XCH - exchange data.
→ swap data b/w A & another operand.

Eg: XCH A, R2 ; exchange A & R2

(7) XCHD - Exchangl lower nibble.
→ swaps only lower 4-bits only

Eg: XCHD A, @R0

(8) SWAP - swap both Low & high nibble.

```
MOV  A, #9Bh   ; A= 10011011b.
SWAP A         ; A=10111001b
```

(26) Bit Manipulation type of instruction of 8051 (8)

→ They are used to set, clear, complement, & move individual bits.

(1) SETB – set
→ used to turn ON a specific bit.

Eg: SETB P1.0 ; set pin P1.0=1
SETB 20H ; set bit at RAM address 20H.

(2) CLR – clear
→ used to turn OFF a specific bit.
Eg: CLR P2.3 ; Clear pin P2.3=0
CLR C ; Clear carry flag.

(3) CPL – complement
Changes 1→0 & 0→1
Eg: CPL P0.5 ; Toggle pin P0.5
CPL C ; Complement carry flag.

(4) ANL C , bit – AND a bit with carry.
Carry = Carry AND bit.
Eg: ANL C, P1.2 ; C=C AND P1.2

(5) ORL C ; bit. OR a bit with carry
carry = Carry OR bit
Eg: ORL C, P3.3 ; C=C OR P3.3

(6) MOV C, bit – move bit into carry.
→ copies the status of bit int carry flag.
Eg: MOV C, P1.7 ; C=P1.7

(7) MOV bit, C – move carry into bit
→ copies the carry flag into specific bit.
Eg: MOV P2.0, C ; P2.0=carry.

(8) JB → Jump if bit = 1

JB P1.0, LABEL ; if P1.0 = 1, jump to LABEL.

(9) JNB → jump if bit = 0

JNB P3.2, LOOP; if P3.2 = 0, jump to LOOP

(10) JBC - jump if bit = 1 & clear it.

Eg: JBC P0.3, AGAIN ; if P0.3 = 1, jump to AGAIN
& AND clear P0.3.

(07) Branching type instruction in 8051

→ Branching instructions are used to change the normal flow of a program.

→ They help in creating loops, decisions, jumps & calls.

Branching instructions are of 3 types:
1. Unconditional Jump
2. Conditional Jump
3. Call & Return instructions.

[1] Unconditional Jump instructions:
→ These instructions always change the program flow.

A. SJMP - short jump

```
SJMP LOOP    ; Jump to Loop
```

B. LJMP - Long Jump

```
LJMP START ; Jump to address START
```

C. AJMP - Absolute Jump

```
AJMP HERE
```

2. Conditional Jump Instructions.

A. JZ Jump if A=0

```
MOV A, R0
eg: JZ ZERO ; If A==0 → Jump
```

B. JNZ Jump if A ≠ 0

```
MOV A, #10
JNZ AGAIN ; Jump because A is non-zero
```

C. CJNE – Compare & jump if not equal

```
CJNE A,#20, NEXT ; if A≠20 → Jump
```

D. DJNZ – decrement & jump if not zero
→ used for loops

```
MOV R1, #5
LOOP: DJNZ R1, LOOP; Repeat 5 times.
```

E. JB – jump if bit = 1 ;

```
JB P1.0, CHECK ; Jump if P1.0 = 1
```

F. JNB – Jump if bit = 0 ;

```
JNB P3.2 , WAIT ; Jump if P3.2 = 0
```

G. JBC – Jump if bit = 1 & clear it.

```
JBC P2.5, AGAIN
```

(3.) CALL instructions

ACALL - Absolute call (2KB range)

ACALL SUB

LCALL - Long call (64KB rangl)

LCALL DELAY ; call delay subroutine

(4.) RETURN instructions;

RET - Return from subroutine

RET ; Returns to main program

RETI - Return from interrupt

RETI ; used inside interrupt service routine.

18] RET & RETI instruction of 8051

① RET – Return from subroutine.

→ RET is used at end of a subroutine
→ It makes the program go back to the instruction after the CALL

WORKING:
① When a subroutine is called using ACALL or LCALL, the return address is stored on the stack
② RET pops that address from stack & returns to main program.

Eg:

```
MAIN:
        MOV A, #20
        ACALL ADD5  ; call subroutine

HERE: SJMP HERE ; stop here

ADD5:
        ADD A ; #5      ; A= A+5
        RET             ; Return to MAIN
```

② RETI - Return from interrupt.

→ RETI is used at the end of an Interrupt
→ It returns program control back to main program & also signals the CPU that interrupt is finished.

Working:
1. Pops return address from stack
2. Clears interrupt-in-progress flag
3. CPU continues main program

Eg:

```
        ORG 0003H          ; external interrupt 0 vector
                             address

INT0_ISR:
        INC R1             ; Example ISR task
        RETI               ; Return from interrupt

MAIN:
        SETB EX0           ; ENABLE INT0
        SETB EA            ; ENABLE global interrupts
HERE: SJMP HERE            ; Main program loop.
```

RET	RETI
used in subroutines	used in interrupt routines
only returns to caller	returns AND clears interrupt flag.
DOES NOT end interrupt cycle	Ends interrupt cycle.

9Q) Instruction in 8051 used for creating a loop in a program

① DJNZ
→ Decrement & jump if not zero

→ Decrements a register or memory location.
→ If result is not zero, the program jumps to the target label
→ If the result becomes zero, it exits the loop.

Eg: MOV R0, #10 ; Load counter

LOOP:
; (Your task here)
DJNZ R0, LOOP; Repeat until R0 becomes 0

R0 = 10 → 9 → 8 → 1
when R0 becomes 0 loop stops.

☺ Requires only one instruction to check, decrement & loop
☺ Very fast
☺ Does not affect the accumulator
☺ Works with registers & direct RAM.

→ Other instructions that loop ① SJMP loop (∞ loop)
② JNZ (loop using accumulator)

→ But DJNZ is the best of all 3.

10) JUMP instructions:

1. Unconditional jumps
2. conditional jumps
3. Bit-based jumps.

(1) UNCONDITIONAL JUMPS: (These jumps happen, without checking any condition)

(a) SJMP – short jump (-128 to +128 bytes).

```
SJMP AGAIN
```

(b) LJMP – long jump (anywhere in 64KB)

```
LJMP START
```

(c) AJMP – Absoulte jump (2KB range)

```
AJMP VALUE
```

(2) CONDITIONAL JUMP: (these jumps happen, if the condition is true)

(a) JZ jump if A=0

```
MOV A,R0
JZ ZERO
```

(b) JNZ jump if A≠0

```
MOV A,#5
JNZ LOOP
```

(c) CJNE compare & jump if not equal

```
CJNE A,#50,NEXT; jump if A≠50.
```

(d) DJNZ - decrement & jump if not zero.

```
MOV R1, #10
LOOP: DJNZ R1, LOOP
```

(3) BIT - RELATED JUMPS;
used to check or manipulate single bits

(a) JB - jump if bit = 1;

```
JB P1.0, ON ; jump if P1.0 = 1
```

(b) JNB - jump if bit = 0;

```
JNB P3.2, WAIT ; jump if P3.2 = 0
```

(c) JBC - jump if Bit = 1 AND clear bit

```
JBC P0.1, AGAIN
```

COMMENTS APNE HISAB SE LEKHO !!!

Q11: Addition of 2 Numbers:

```
MOV  A, 30H ;
ADD  A, 31H ;
MOV  32H, A ;

HERE: SJMP HERE; stop the program
```

Q12 Subtraction of 2 Numbers:

```
CLR C    ;
MOV A, 30H ;
SUBB A, 31H ;
MOV 32H, A;

HERE: SJMP HERE; stop the progeram
```

Q13 Multiplication of 2 number:

```
MOV A, 30H ;
MOV B, 31H ;
MUL AB     ;
MOV 32H, A ;
MOV 33H, B ;

HERE: SJMP HERE; stop the program.
```

4) Transfer Block of 5 numbers;

```
MOV R0, #30H ;
MOV R1, #40H ;
MOV R2, #05H ;

LOOP:
    MOV A, @R0 ;
    MOV @R1, A ;

    INC R0      ;
    INC R1      ;
   DJNZ R2, LOOP;

HERE : SJMP HERE ; stop program
```

D Y PATIL UNIVERSITY RAMRAO ADIK

Ramrao Adik Institute of Technology, Nerul, Navi Mumbai

M-4 ES

Question Bank – 4 (Module-4)

Academic year: 2025-2026

Subject: Embedded System Class: S.E. Sem : III

1. List and explain all addressing modes in 8051 microcontroller.
2. Explain how I/O ports are initialized in 8051 with suitable instructions and examples.
3. Explain MUL and DIV instructions in 8051 with suitable examples.
4. Explain ANL, ORL and XRL instructions in 8051 with suitable examples.
5. Explain RL, RLC, RR, RRC instructions in 8051 with suitable examples.
6. Explain TMOD and TCON SFR's of 8051 microcontroller in detail.
7. Explain how 8051 can be used in Counter Mode.
8. Explain how 8051 serial communication works with suitable SFR's.
9. Explain what RS232 is and hence explain DTE and DCE too.
10. Explain the 8051 interrupts with Interrupt Vector table (IVT) in detail.
11. Write a assembly language program to find the largest number from given five numbers.
12. Write a assembly language program to unpack a packed BCD Number.
13. Write a assembly language program to generate a delay of 1ms.
14. Write a program for the 8051 to transfer letter "A" continuously on the serial port.
15. Write a program to detect the position of switch connected to P2.7 pin and accordingly ON and OFF the LED connected at pin P1.0. LED should be ON when the switch is in ON position and LED should be OFF when the switch is in OFF position.

Addressing modes in 8051 microcontroller

→ Addressing modes tell the CPU where the data/operand is located.

(1) Immediate Addressing mode:

→ Data is given immediately in the instruction

Syntax:

```
MOV A,#25H
```

Load A with value 25H

→ Used for constants.

(2) Register Addressing Mode:

→ Operand is inside R0-R7

Syntax

```
MOV A,R2
MOV R5,A
```

move data b/w A & registers.

→ Fastest mode.

(3) Direct Addressing Mode:

→ Operand is in internal RAM or SFR using its direct address.

Syntax:

```
MOV A,30H
MOV 40H,A
```

→ Easy access RAM.

④ Indirect Addressing Mode:

→ Register R0 or R1 holds the address of data.

Syntax;

```
MOV R0, #30H
MOV A, @R0
```

R0 = address

@R0 = data at the address

→ Used for arrays, tables, block transfer.

⑤ Indexed Addressing Mode

→ Used to access code memory (ROM) using DPTR+A.

Syntax:

```
MOV DPTR, #TABLE
MOVC A, @+DPTR
```

fetch from lookup table stored in program memory

→ Used for lookup tables, constants

⑥ Bit addressing Mode:

→ Used for accessing single bits of RAM or SFRs.

Syntax;

```
SET P1.0
CLR 20H
JB P3.2, LABEL
```

→ operate on individual bits only

→ used for bit-level control, flags, ports.

Q2) I/O ports are initialized in 8051

→ 8051 has 4 ports: P0, P1, P2, P3.
→ Each port can work as i/p or o/p.

(1) Initializing Port as o/p:
→ To make a port work as o/p, you simply write any value to it.
Eg:

```
MOV P1, #00H  ; P1 becomes o/p (all pins =0)
MOV P2 #0FFH  ; P2 o/p & all pins =1
MOV P3, #55   ; send 55H to P3
```

→ In 8051, when u write a port → it act as o/p.

(2) Initializing Port as i/p:
→ To make a port i/p, you must write 1 to that port.
Eg:

```
MOV P0, #0FFH ; Make Port 0 i/p
MOV P3, #0FFH ; Make Port 3 i/p
```

Special case: Port 0.
→ Port 0 has no internal pull-ups.
→ for i/p or o/p, it needs external pull-ups resistors.
Eg:

```
MOV P0, #0FFH ; P0 as i/p (with external pull-ups).
```

3. Bit-level initialization of Ports:

Set a single pin:

```
SETB P1.0 ; Make P1.0 = 1
```

Clear a pin:

```
CLR P2.5 ; make P2.5 = 0
```

Read a pin:

```
MOV C, P3.2 ; Read switch at P3.2 into carry flag
```

write using carry:

```
MOV P1.7, C ;
```

4. Simple PROGRAM;

Eg:

```
MOV P1, #55H
```

Eg:

```
MOV P3, #0FFH ;
MOV A, P3 ;
MOV P1, A ;
```

Eg:

```
BACK:
    CPL P1.0
    SJMP BACK
```

037

MUL AB

→ Lower 8 bits → stored in A

→ Upper 8 bits (overflow) → stored in B

→ Carry flag = 0

overflow = 1 if result > 255

Eg: MOV A, #05H
MOV B, #04H
MUL AB

DIV AB

→ A ÷ B

Quotient → stored in A

Remainder → stored in B

overflow = 1 if B = 0.

Eg:

MOV A, #20H
MOV B, #06H
DIV AB

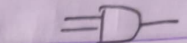
Q44) ANL, ORL & XRL instructions in 8051

These are 3 logical instructions used to perform AND, OR & XOR operations.

(1) ANL - AND logical

→ AND = keeps only common 1s used to clear specific bits (masking)

Eg: MOV A, #0F0H
ANL A, #0F8H

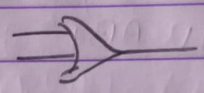


→ AND both values bit by bit.
→ clear / select certain bits.

(2) ORL - OR logical

→ OR = sets bits to 1 when either bit is 1 used to turn ON specific bits

Eg:
MOV A, #33H
ORL A, #0F0H

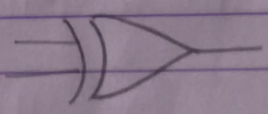


→ OR both values → sets more bits to 1.
→ Set / turn ON bits

(3) XRL - XOR logical

→ XOR = 1 only when bits are different used to toggle bits

Eg:
MOV A, #55h
XRL A, #0FFH



→ XOR with FF → inverts all bits.
→ Toggle / Invert bits.

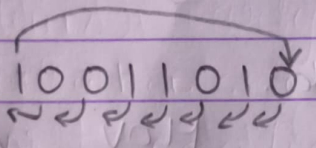
Q5) RL, RLC, RR, RRC instructions in 8051.

→ These are 4 rotate instructions used on the Accumulator (A).
→ Think of bits rotating like shifting them in a circle.

(1) RL A – Rotate left (No carry used).

→ All bits move left by 1
→ The leftmost bit comes to the rightmost place
→ CARRY is NOT affected.

Eg: MOV A, #10011010B
RLA
o/p 00110101B



(2) RLC A – Rotate left through carry.
→ Bits move left, But leftmost bit goes to carry
→ Old carry enters rightmost bit

Eg: MOV A, #10011010B
SETB C
RLC A
o/p · A = 0 0110101 B
· C = 1

③ RRA → Rotate Right (No carry)
→ Bits move right by 1
→ Rightmost bit goes to leftmost.
→ Carry not affected.

Eg: MOV A, #10011010B
RR A

O/P 01001101B

④ RRC A - Rotate Right Through Carry:

→ Bits move right
→ rightmost bit → carry
→ Old carry → left most bit.

Example:

MOV A, #10011010B
CLR C
RRC A

O/P A= 01001101
C =0

RL — left rotate
RLC — left rotate through carry.
RR — right rotate
RRC — right rotate through carry.

Q6) TMOD & TCON SFRs of 8051:

→ Timers in 8051 are controlled by 2 special registers.

- TMOD → selects Mode of timer
- TCON → controls start/stop and flags.

(1) TMOD – Timer MOD Register (Address: 89H)

→ TMOD selects HOW Timer0 & Timer1 will work.

GATE	C/T	M1	M0	(Timer1)
GATE	C/T	M1	M0	(Timer0)

GATE:

→ 0 → timer runs normally
1 → timer runs only if external pin INT0/INT1 = 1

C/T:

→ 0 → Timer (counts internal clock)
1 → counter (counts external pulses).

	Mode	Bits	Meaning.
(1)	Mode 0	00	13-bit timer
(2)	Mode 1	01	16-bit timer (most used)
(3)	Mode 2	10	8-bit auto-reload.
(4)	Mode 4	11	split timer mode.

Eg: MOV TMOD, #01H

② TCON - Timer control register:
(Address : 88H)

→ Used to start / stop timers & track overflow.

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0

- TR0 / TR1 - Timer run bits.
  1 → start
  0 → stop

```
SETB TR0 ; start timer 0
CLR  TR1 ; stop  timer 1
```

- TF0/TF1 - Timer overflow flags:
→ Set to 1 when timer overflows
→ Must be cleared in software.

```
JNB  TF0, HERE ; wait for overflow
CLR  TF0       ; clear flag.
```

- IT0 / IT1 - Interrupt type bits:

  1 → Edge triggered
  0 → level triggered.

- IE0 / IE1 - External interrupt flags.

Q7) Counter Mode in 8051

→ 8051 timers can works as counters to count external pulses instead of time.

→ Counter mode is selected using the C/T bit in TMOD register.

How to use 8051 as counter:

S1: Select counter Mode

Set C/T = 1 in TMOD:

Eg: MOV TMOD, #05H

S2: Apply external pulses:

→ External pulses must be applied at:
T0 pin (P3.4) → for counter 0
T1 pin (P3.5) → for counter 1

each pulse increments the counter.

S3: Start the count:

eg: SETB TR0; Star counter 0.

S4: Check for overflow:

→ when counter reaches its max value (FFFFH for Mode 1)
overflow flag TF0=1.

```
JNB TF0, $      ; wait until overflow
CLR TF0         ; clear overflow
                flag
```

Eg:

```
MOV TMOD, #05H ; counter0, Mode1 (16-bit)
SETB TR0       ; start counter
HERE:
    JNB TF0, HERE ; wait for overflow
    CLR TF0
    SJMP HERE
```

Q8 Serial communication in 8051

→ 8051 uses UART for serial communication
3 main registers control it:

① SCON (serial control register):
→ Controls serial mode & TX/RX.
Important bits:

SM0	SM1	SM2	REN	TB8	RB8	TI	RI

SM0, SM1 → select serial mode
REN → Enable serial reception
TI → Transmit interrupt flag
RI → Recieve interrupt flag

② SBUF (serial Buffer):
→ Used to send/recieve data.

```
Send:    MOV SBUF, #'A'
receive: MOV A, SBUF
```

③ Baud Rate (speed) via Timer 1:
→ Timer 1 is set in Mode2 to generate baud rate

Example (9600 baud):

```
MOV TMOD, #20H
MOV TH1, #0FDH
SETB TR1.
```

Example;

→ ENABLE UART:

```
MOV SCON, #50H ; Mode 1, REN=1
```

→ Set Baud rate (Timer 1):

```
MOV TMOD, #20H
MOV TH1, #0FDH
SETB TR1
```

→ Send data:

```
MOV SBUF, #'A'
JNB TI, $
CLR TI
```

→ Receive data:

```
JNB RI, $
MOV A, SBUF
CLR RI
```

SCON controls serial mode
SBUF holds data.
Timer 1 sets baud rate;

Q9) What is RS-232?
DTE & DCE;

RS-232:
RS-232 is a serial communication standard used to ~~control~~ connect computers & devices.

It defines:
→ voltage level
→ Pins
→ Baud rate
→ How data is sent serially

Used in modems, old PCs, microcontrollers.

⊖ Voltage level:
+3V to +15V = 0 (logic LOW)
-3V to -15V = 1 (logic HIGH)

DTE (Data Terminal equipment)
→ Devices that send data OUT.
→ Eg: Computer / Microcontroller
TX pin → O/P
RX pin → I/P.

DCE (Data communication equipment)
→ Devices that ~~send data~~ recieve data first
Eg: Modem.
TX pin → I/P
Rx pin → O/P

10) Interrupts with IVT (interrupt vector table) in 8051.

→ 8051 has 5 interrupts

→ Each interrupt has a fixed vector address where the ISR (interrupt service routine) starts.

→ List of 8051 interrupts.

Interrupt	Name	Vector Address	Priority
INT0	External interrupt 0	0003H	low
T0	Timer 0 overflow	000BH	Low
INT1	External interrupt 1	0013H	high
T1	Timer 1 overflow	001BH	high
Serial	serial port interrupt	0023H	Lowest

→ IVT

Address	Interrupt
0000H	Reset
0003H	INT0 ISR
000BH	Timer0 ISR.
0013H	INT1 ISR
001BH	Timer1 ISR
0023H	Serial ISR

→ How interrupts work:

1. Interrupt occurs
2. 8051 jumps to its vector address
3. Executes the ISR
4. Returns using RETI.

→ Interrupt Enable (IE): (IE register)

- used to turn ON / OFF interrupts

EA = 1 → enable all interrupts.
EX0 = 1 → Enable INT0.
ET0 = 1 → Enable Timer0.

Example; MOV IE, #81H ; EA=1, EX0=1

→ Interrupt priority → IP register.
· Decides which interrupt runs first.

0 → low priority
1 → high priority

~~Example 1 of 8051~~

Q11) Largest of 5 numbers:

```
        MOV   R0, #30H    ; R0 points to 1st number
        MOV   R1, #05     ; counter=5 numbers
        MOV   A, @R0      ; Assume 1st number is
                            largest.
        INC   R0
LOOP:
        MOV   B, @R0      ; Read next number
        CJNE  A,B, CHEK
        SJMP  NEXT

CHECK:
        JC    NEXT        ; if A<B then B is larger
        MOV   A, B        ; update largest number.

NEXT :
        INC   R0
        DJNZ  R1, LOOP

MOV   40H , A     ; store largest number
HERE: SJMP HERE
```

Q12) ALP to unpack a packed BCD number:

```
MOV A, 30H          ; Get packed BCD
ANL A, #0F0H        ; mask upper nibble
SWAP A              ; Bring upper nibble to lower
MOV 40H, A          ; store upper digit

MOV A, 30H          ; load packed again
ANL A, #0FH         ; Mask lower nibble
MOV 41H, A          ; store lower digit.

HERE: SJMP HERE.
```

Q13) AP to generate a delay of 1 ms.

```
ORG 0000H

MAIN:
    CALL DELAY      ; call 1ms delay
    SJMP MAIN       ; Repeat forever.

;-----------------------
; 1 ms Delay Subroutine.
;-----------------------

DELAY:
    MOV R2, #250    ; 250 x 4 cycles = ~1000 us = 1ms

HERE:
    DJNZ R2, HERE
    RET

END.
```

Q14) Program to detect switch at P2.7 & ON/OFF LED at P1.0.

```
MOV P2, #0FFH   ; make port 2 i/p
MOV P1, #00H    ; make port 1 o/p

MAIN:
    JB P2.7, LED_ON;   If P2.7 = 1 -> Jump to
                       LED_ON

LED_OFF:
    CLR P1.0           ; LED OFF
    SJMP MAIN

LED_ON:
    SETB P1.0          ; LED ON.
    SJMP MAIN
```

D Y PATIL UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Ramrao Adik Institute of Technology, Nerul, Navi Mumbai

M-5 / ES

Question Bank – 5 (Module-5)

Academic year: 2025-2026

Subject: Embedded System Class: S.E. Sem : III

1. Draw detailed diagram of Seven Segment Display (SSD) interfacing with 8051 microcontroller and write suitable program for the same.
2. Discuss in detail the Embedded case study of "Digital Camera".
3. Discuss in detail the Embedded case study of "Washing Machine".
4. Draw detailed diagram of DC Motor interfacing with 8051 microcontroller and write suitable program for the same.
5. Draw detailed diagram of Stepper Motor interfacing with 8051 microcontroller and write suitable program for the same.
6. Draw detailed diagram of Liquid Crystal Display (LCD) interfacing with 8051 microcontroller and write suitable program for the same.
7. Draw detailed diagram of Hex Keyboard interfacing with 8051 microcontroller and write suitable program for the same.
8. Explain what is "Wave Drive, Full Step and Half Step" sequence with reference to stepper motor interfacing.
9. Assume that switch SW is connected to the pin P2.0 and P1.0 is connected to one of the input of DC Motor and P1.1 is connected to another input of DC Motor. Write a program to monitor the status of SW, if SW = 1, rotate the dc motor in a clockwise direction, or if SW = 0, reverse the direction of the motor.
10. Explain the difference between Common Cathode and Common Anode SSDs.

11 SSD → seven segment display.
SSD interfacing with 8051

→ Program to display digit "5" on SSD

```
        ORG   0000H

        MOV   P1, #00H   ; clear port
        MOV   A, #6DH    ; Hex code of digit 5
        MOV   P1, A      ; send to SSD

HERE :  SJMP  HERE;      ∞ loop

        END
```

→ Program to display no. 1-9 continuously:

```
        ORG   0000H
NUMBERS: DB
 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH

        MOV   DPTR, #NUMBERS  ; table of digits.

MAIN:
        MOV   R0, #10         ; 10 digits.

LOOP:
        MOVC  A, @A+DPTR      ; Read pattern
        Mov   P1, A           ; o/p to SSD
        ACALL DELAY           ; small delay
        INC   DPTR            ; small delay
        DJNZ  R0, loop

        MOV   DPTR, #NUMBERS  ; Reload Table
        SJMP  MAIN            ;
DELAY:
        MOV   R2, #255
D1 :    DJNZ  R2, D1
        RET
        END
```

CASE STUDY:

Q2) Digital camera

→ Components:

1. Lens system
2. Image Sensor (CCD / CMOS)
3. Microcontroller / DSP
4. Memory (Flash card)
5. Display unit (LCD screen)
6. User interface
7. Battery & power control.

→ Working:

S1 Press Capture Button
S2 Light passes through lens.
S3 Sensor converts light to electrical signal
S4 Microcontroller processes image
S5 Image stored in memory
S6 Display preview on LCD.

→ Why it is an ES?

- It has a dedicated function: capture images.
- Uses microcontroller + sensor + memory
- User cannot reprogram it.
- Real-time processing required
- Low power, compact, automatic
- Fully self-contained system.

CASE STUDY:

Q3) Washing Machine

→ It is a real-life E.S. used to wash clothes automatically.

Main Components:
- Microcontroller: controls all operations
- Motor → rotates drum
- Water level sensor → checks water amount
- Temperature Sensor → control heater.
- Valves → water IN/OUT control.
- Keypad / Buttons → user selects mode
- Display LEDs / LCD → show status
- Buzzer → end-cycle alert.

Working:
1. User selects wash mode
2. Microcontroller opens water valve → drum filled.
3. Motor rotates drum clockwise/anticlockwise for washing
4. Heater warms water (if required)
5. Drain valve opens → water removed.
6. Spin cycle starts → motor runs at high speed
7. Machine stops & alerts user (buzzer).

Why it is an ES?
→ single dedicated function: wash clothes
→ uses microcontroller for all tasks
→ Real-time operation
→ user cannot reprogram it.
→ Works automatically with sensors + actuators.

7) DC motor interfacing with 8051 (L293D driver)

Program:

① Program to rotate motor clockwise

```
MOV P1, #01H ; P1.0=1, P1.1=0 → clockwise
SJMP $
```

② Program to rotate motor anticlockwise

```
MOV P1, #02H ; P1.0=0, P1.1=1 → anticlockwise
SJMP $
```

③ Program to rotate in both directions with delay:

```
MAIN:
    MOV P1, #01 H; clockwise
    ACALL DELAY

    MOV P1 # 02H ; anticlockwise
    ACALL DELAY
    SJMP MAIN

DELAY:
    MOV R2, # 255
D1: DJNZ R2, D1
    RET
```

→ DC motor is interfaced to 8051 using L293D driver
→ P1.0 & P1.1 control motor direction
→ 01H → clockwise, 02H → anticlockwise.

4 Stepper Motor interfacing with 8051

Stepper motor sequence:

Step	P2.3	P2.2	P2.1	P2.0	Coil
1	1	0	0	0	A
2	0	1	0	0	B
3	0	0	1	0	C
4	0	0	0	1	D

This rotates the motor clockwise.

Program: Clockwise Rotate.

```
        ORG  000H
        MOV  P2, #00H ;   Clear port
MAIN:
        MOV  A, #88H ;    1000  (Coil A)
        MOV  P2, A
        ACALL  DELAY

        MOV  A, #44H      ; 0100  (Coil B)
        MOV  P2, A
        ACALL  DELAY

        MOV  A, #22H      ; 0010  (Coil C)
        MOV  P2, A
        ACALL  DELAY

        MOV  A, #11H      ; 0001 (Coil D)
        MOV  P2, A
        ACALL  DELAY

        SJMP  MAIN
```

```
DELAY:
    MOV R2 # 255
D1:  DJNZ  R2,D1
     RET
END
```

- Anticlockwise sequence:

D → C → B → A
0001 → 0010 → 0100 → 1000

- Stepper motor is interfaced using ULN2003 driver
- 8051 sends step sequence through P2.0 – P2.3
- Energizing coils in order moves the motor in steps.
- Reverse sequence → reverse rotation.

Diagram:

LCD interfacing with 8051

→ We use a 16×2 LCD in a 8-bit mode with 8051.

→ Important LCD commands:

Command	Hex	Function
Clear display	01H	Clears LCD
8-bit, 2-line mode	38H	Function set
Display ON	0CH	show characters
Cursor Hom	80H	first line start

→ Hello on LCD.

```
ORG 0000H ; P2=data, P3.5=RS, P3.6=RW, P3.7=EN

MAIN:
    ACALL INIT

    MOV A, #'H'
    ACALL WRITE_DATA

    MOV A, #'E'
    ACALL WRITE_DATA

    MOV A, #'L'
    ACALL WRITE_DATA

    MOV A, #'L'
    ACALL WRITE_DATA

    MOV A, #'O'
    ACALL WRITE_DATA.

HERE: SJMP HERE

;-----------------------
; Initialize LCD
;-----------------------
```

```
INIT:
    MOV A, #38H
    ACALL WRITE-CMD
    MOV A, #04H
    ACALL WRITE-CMD
    MOV A, #01H
    ACALL WRITE-CMD
    RET

;-----------------------
; send command.
;-----------------------
WRITE-CMD:
        MOV P2, A
        CLR P3.5        ; RS = 0
        CLR P3.6        ; RW = 0
        SETB P3.7       ; EN = 1
        CLR P3.7        ; EN = 0
        ACALL DELAY
        RET

;-----------------------
; send data.
;-----------------------
WRITE-CMD:
        MOV P2, A
        SETB P3.5       ; RS = 1
        CLR P3.6        ; RW = 0
        SETB P3.7       ; EN = 1
        CLR P3.7        ; EN = 0
        ACALL DELAY     ;
        RET

;-----------------------
DELAY:
    MOV R2, #255
D1: DJNZ R2, D1
    RET
END.
```

Q7.) Hex Keyboard interfacing with 8051;

→ It has 16 keys arranged in 4 rows & 4 columns.

→ Working
- Send 0 to each row one-by-one
- Read columns
- If any columns becomes 0, that key is pressed
- Use key map to identify which key it is.

→ Program:

```
        ORG   0000H
MAIN:
        MOV   P1, #0F0H ; upper nibble i/p (cols),
                          lower nibble o/p (rows)
ROW 0:
        MOV   P1, #0FEH ; ROW 0=0
        JB    P1.4 ; ROW1   ; if C0=0 → key 1 pressed.
        MOV   A, #'1'
        SJMP  DONE

ROW1:
        MOV   P1, #0FDH
        JB    P1.4, ROW2
        MOV   A,#'4'
        SJMP  DONE

ROW2:
        MOV   P1, #0FBH
        JB    P1.4, ROW3
        MOV   A, #'7'
        SJMP  DONE

ROW 3:
        MOV   P1, #0F7H
        JB    P1.4, ROW0
        MOV   A,#'E'
        SJMP  DONE
```

DONE:
SJMP MAIN ; contains pressed key.
END

Diagram:

Q8} Wave Drive, Full step & Half step sequence (stepper Motor):

→ It rotates by energizing coils in different patterns
→ There 3 typs of stepping modes:

① Wave drive: (single-coil excitation)
→ only ONE coil ON at a time
→ least torque
→ smooth but weak movement

Sequence:

A → B → C → D

Binary: 1000
0100
0010
0001

P2.3, P2.2, P2.1, P2.0

② FULL STEP: (Two-Coil Excitation)
→ Two coils ON at a time
→ More torque
→ Faster movement

Sequence;

AB → BC → CD → DA

Binary;

1100
0110
0011
1001

Half step: (one + 2 coil alternating):
→ combination of wave + full step
→ highest resolution (small of steps)
→ smoother movement

Sequence;

A → AB → B → BC → C → CD → D → DA

Binary;

1000
1100
0100
0110
0010
0011
0001
1001

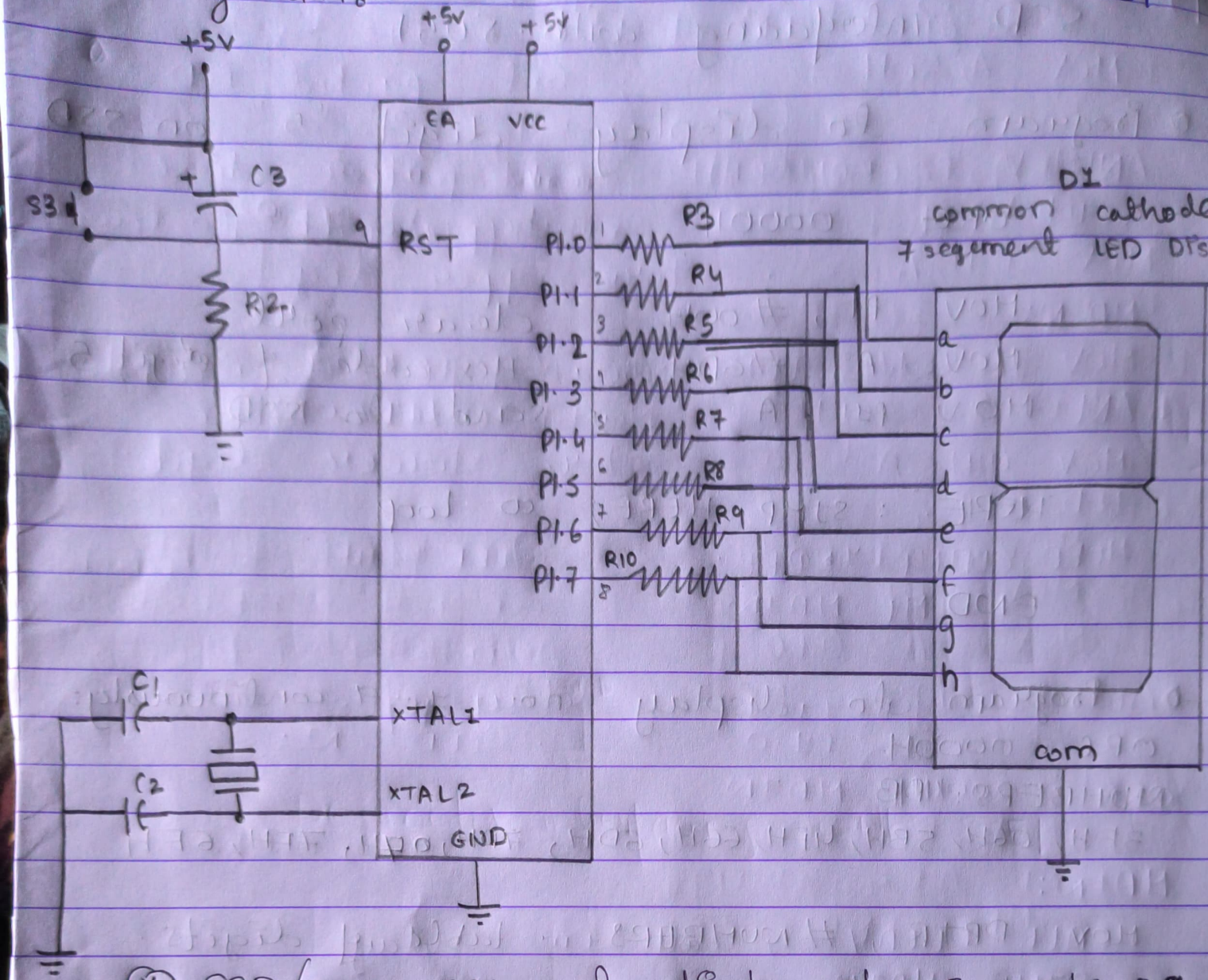
Q9) Program to control DC Motor direction using switch (P2.0):

```
        ORG   0000H
MAIN:
        JB  P2.0, CW;  If P2.0=1 → Jump to CW

ACW:                ; anticlockwise
        CLR  P1.0
        SETB P1.1
        SJMP  MAIN

CW:                 ; clockwise
        SETB  P1.0
        CLR   P1.1
        SJMP  MAIN

        END.
```

Common Cathode (CC)	Common Anode (CA)
① All cathodes tied to GND	All nodes tied to +5V
② Segments ON with logic 1	segments ON with logic 0
③ Segments OFF with logic 0	segments OFF with logic 1
④ uses active-high logic	uses active-low logic
⑤ easy to interface with 8051	Needs inverted logic for displaying
⑥ Microcontroller sources current	Microcontroller sinks current
⑦ Hex codes used are direct	Hex codes need inversion.
⑧ low internal resistance	higher internal resistance.
⑨ Commonly used in bobby projects	common in multiplexed displays.
⑩ Simpler wiring	Slightly more complex wiring
⑪ LED segment lights when 1 is applied	LED segment lights when 0 is applied.
⑫ Good for beginners	Requires care in programming.

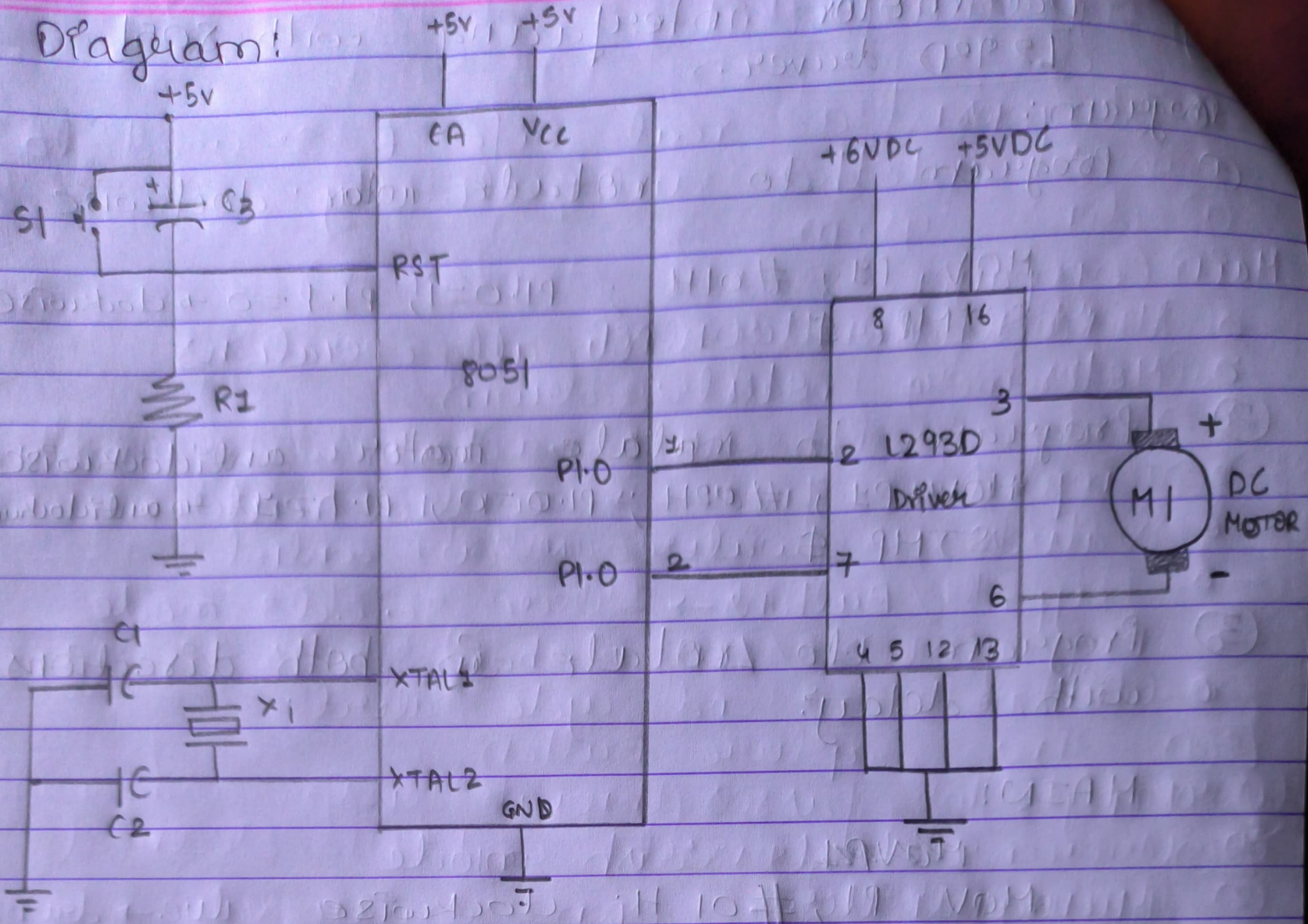
GND

Diagram: SSD with 8051



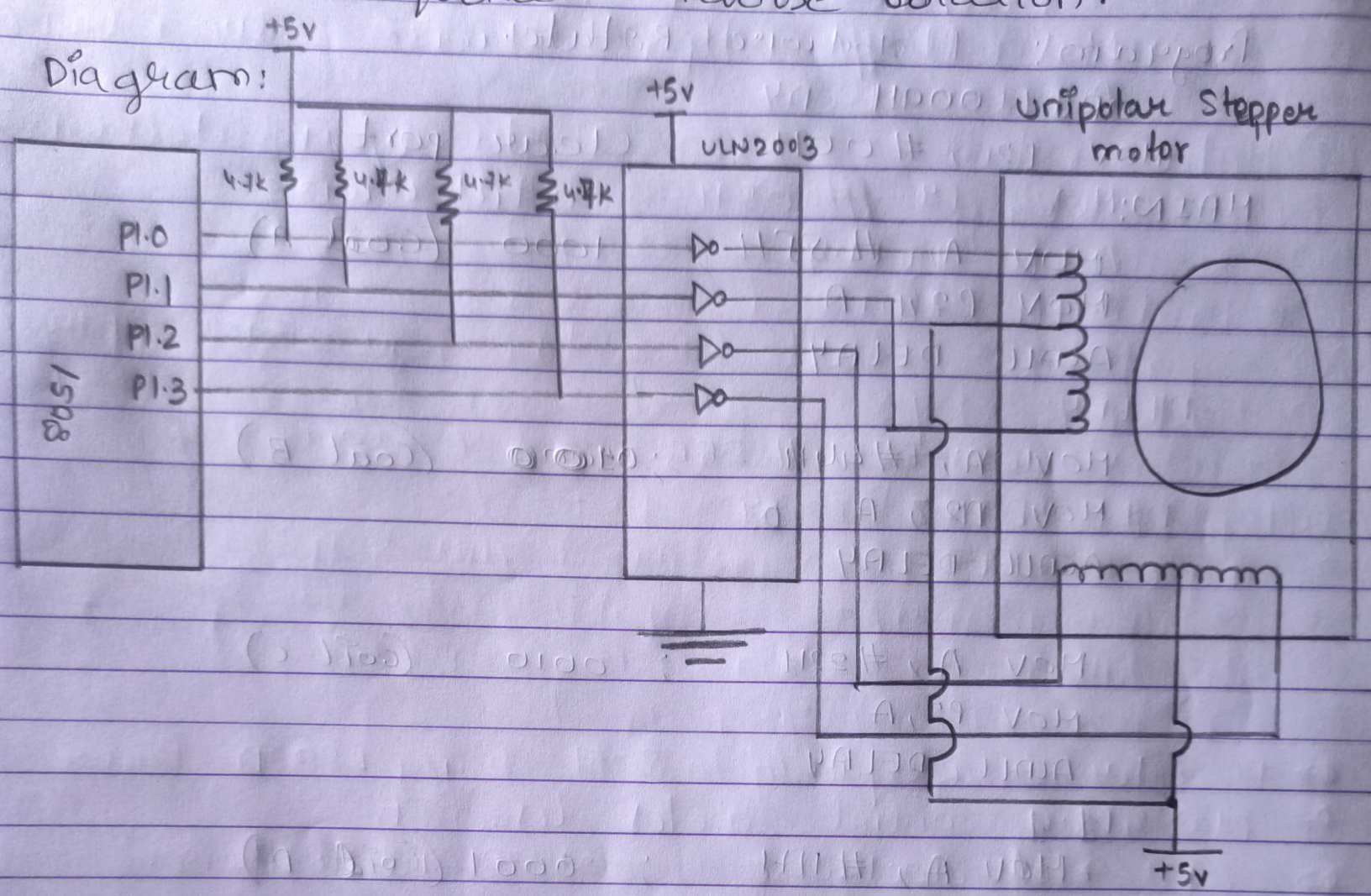
① SSD (seven segment display shows number 0-9 using

Diagram:



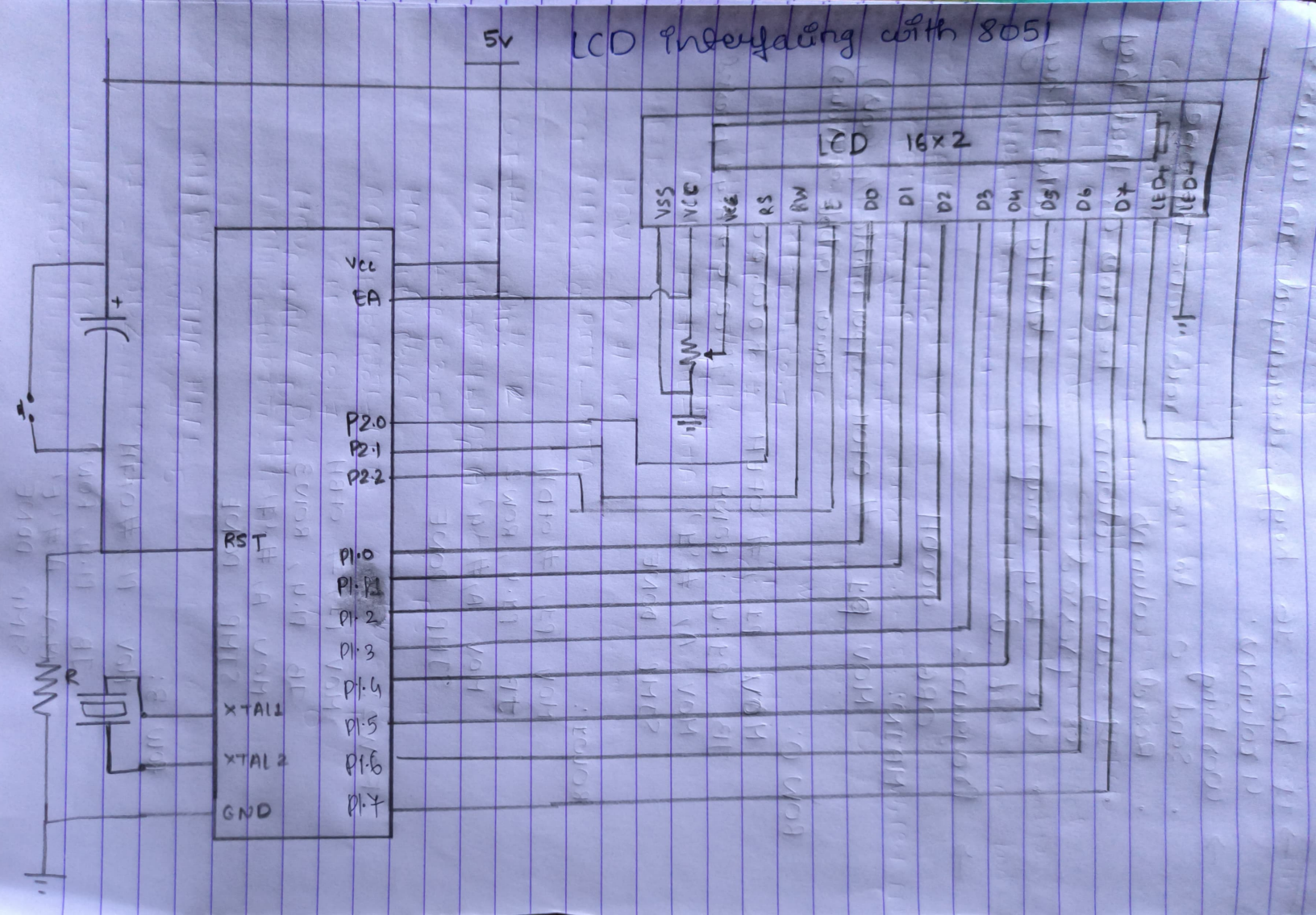
DC motor interfacing with 8051

Diagram:



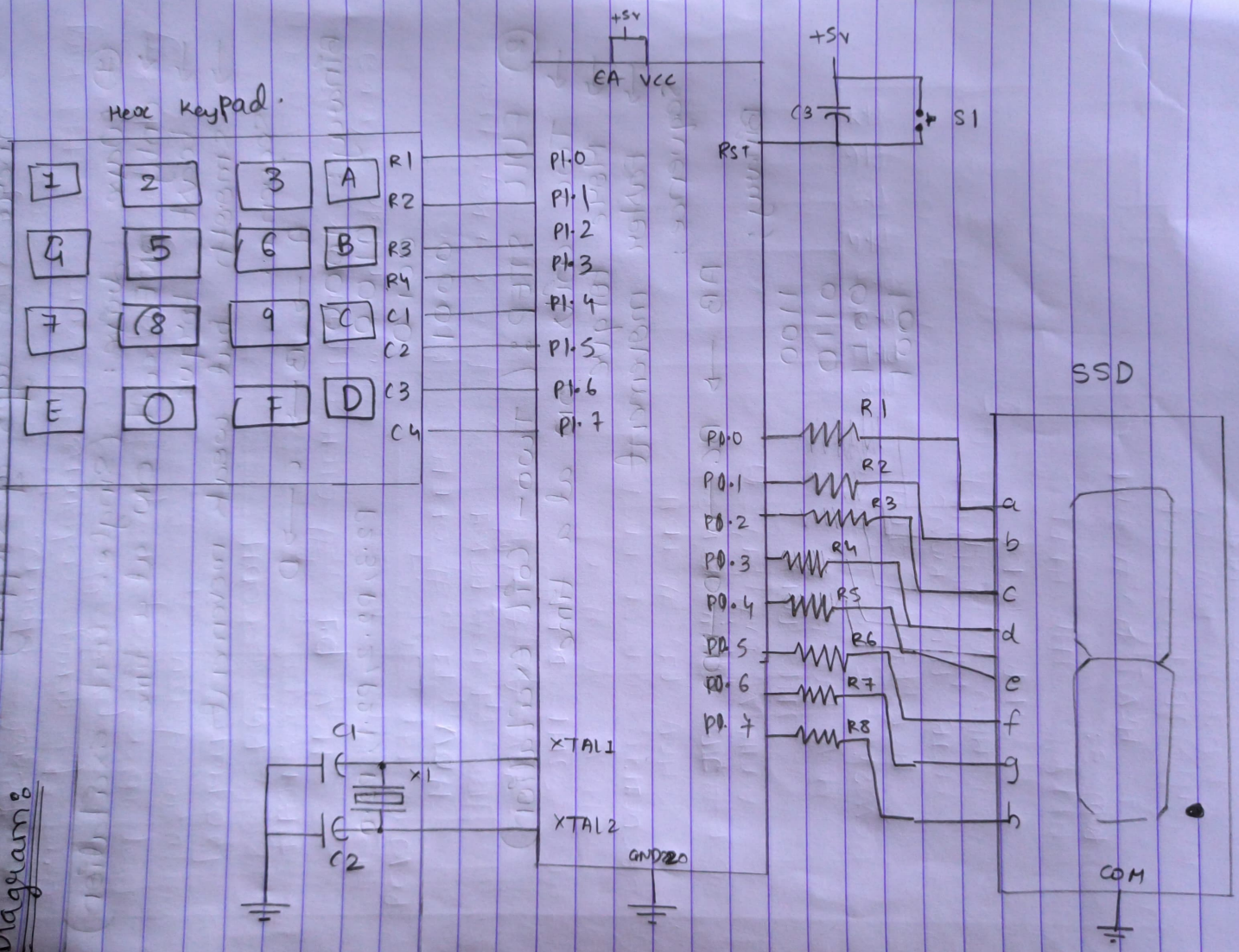
Stepper Motor interfacing with 8051

LCD Interfacing with 8051



Interfacing Hex Keypad to 8051

Diagram:



D Y PATIL UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY, NAVI MUMBAI

Ramrao Adik Institute of Technology, Nerul, Navi Mumbai

Question Bank – 6 (Module-6)

Academic year: 2025-2026

M-6 ES

Subject: Embedded System Class: S.E. Sem : III

1. Explain the main features of ARM architecture.
2. Draw the ARM architecture and discuss the main blocks.
3. Discuss the General-Purpose Register set and CPSR in ARM architecture.
4. Describe the different operating modes in ARM architecture.
5. Explaining how pipelining process in ARM architecture works?
6. Write a short note on Cortex processors from the family of ARM CPU cores.
7. Elaborate the features of Cortex ARM processors.
8. Explain three stage pipelining process in ARM architecture.
9. Explain what Cortex-A, Cortex-R and Cortex-M in ARM architecture is.
10. What are the general features of Cortex ARM processors?

Q1] Main features of ARM architectures:

1. RIS Architecture
2. 32-bit architecture
3. load- store design
4. large Register
5. Thumb instruction set
6. Pipelining
7. Conditional execution
8. low power consumption
9. Multiple operating mode
10. On - chip debugging
11. Barrel shifter
12. Integrated DSP instructions in Cortex series.

ARM is a 32-bit RISC processor.

Q2] Main Blocks of ARM architecture:

① Register File
→ contains 16 general purpose registers R0-R15
→ special registers: PC (R15), LR (R14), SP (R13)
→ also includes CPSR & SPSR.

② ALU
→ Performs addition, subtraction, AND, OR, XOR
→ works with barrel shifter for fast operations

③ Barrel Shifter:
→ Allows shift + ALU operation in one cycle.
→ Improves speed of data processing

④ Instruction Pipeline:
→ ARM uses 3-stage or 5-stage pipeline
→ stages: Fetch → Decode → Execute
→ Increases instruction throughput.

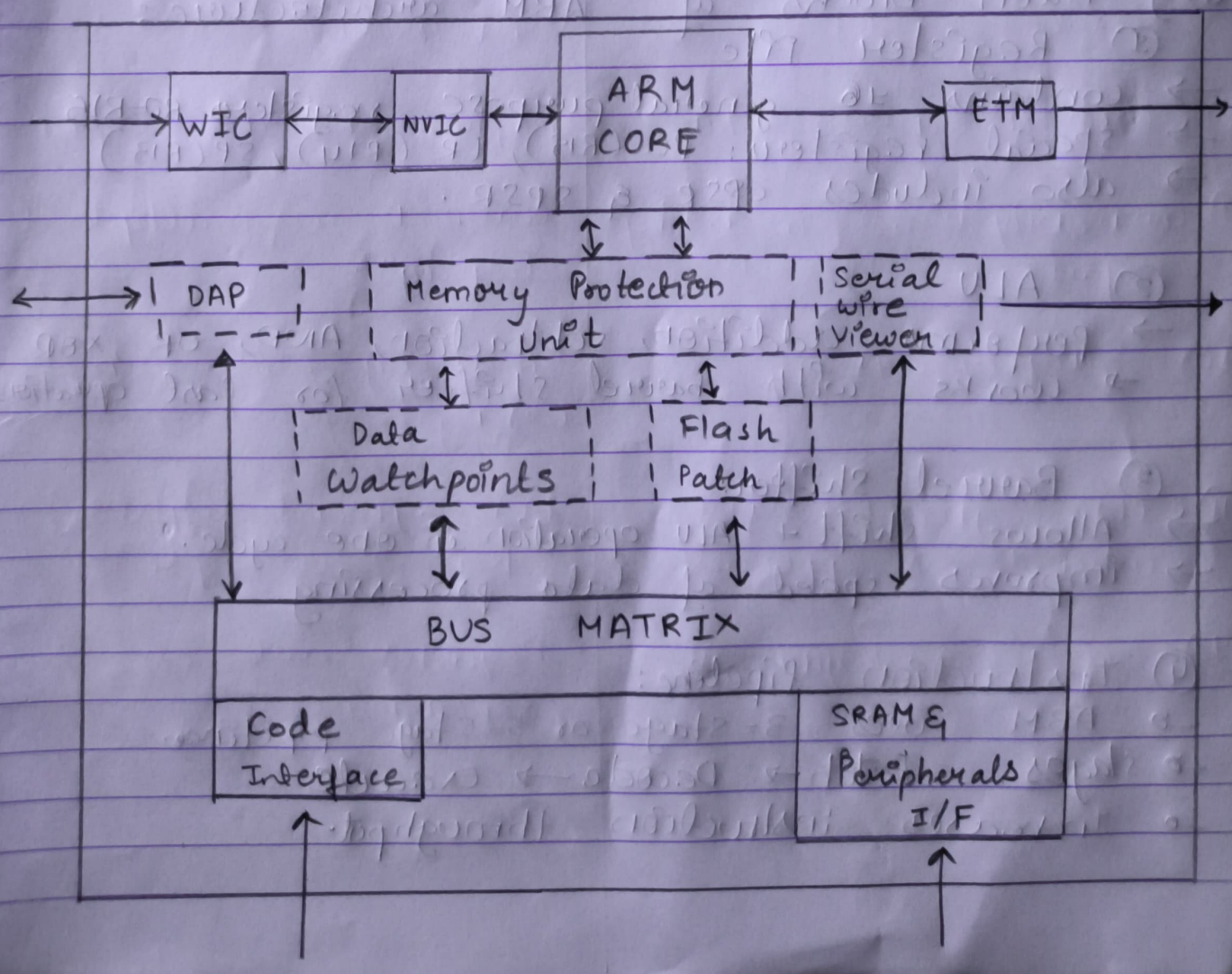
⑤ Load-Store unit:
→ only LOAD & STORE instructions access memory
→ All other operations use registers.
→ Makes ARM a true RISC CPU.

⑥ Memory Interface:
→ Connects CPU to instruction/data memory
→ Controls address & data bus.

⑦ Bus interface unit
→ communicates with external peripherals
→ controls system buses.

⑧ Exception / Interrupt Control:
→ Handles IRQ, FIQ, SWI, reset
→ supports nested & fast interrupt response.

Diagram:



Q3) General-Purpose Register set & CPSR in ARM architecture

(1) General-Purpose Registers (R0-R15):

→ ARM has 16 main registers (32-bit each).

R0-R12 → general-purpose

- used for data storage, arithmetic, temporary values
- very fast access
- Reduce need for memory access.

R13 → SP (Stack Pointer)

- Points to top of stack
- used during subroutines, interrupts, context switching.

R14 → LR (Link Register)

- Stores return address during subroutine calls
- Used by BL (Branch with Link).

R15 → PC (Program Counter)

- Holds address of current instruction
- Always 2 instructions ahead due to pipeling.

(2) CPSR (Current Program Status Register):

→ It stores, flags, operating mode, & interrupt status.

Field	Meaning
N	-ve flag
Z	zero flag
C	carry flag
V	overflow flag
I	IRQ disable
F	FIQ disable
T	Thumb state. (1 = thumb, 0 = ARM)

Mode bits:
→ selects CPU mode

③ SPSR (saved program status Register):
→ used in exception modes to store previous CPSR so that CPU can return after interrupt/exception.

OR

83} General-Purpose registers:
ARM has 16 registers (R0-R15):
- R0-R12 → General-Purpose
- R13 (SP) → Stack Pointer
- R14 (LR) → Link Register (return address)
- R15 (PC) → Program counter.

CPSR (current Program Status Register)
- Flags → N, Z, C, V
- Interrupt bits → I (IRQ disable), F (FIQ disable)
- Mode bits → select CPU mode
- T bit → ARM / thumb state.

SPSR
- stores old CSPR during exceptions.

Q.4) ARM operating Modes:

→ ARM processors support 7 operating modes
→ These modes help in task switching, exception handling & protection.

① User mode
→ Normal program execution mode for applications
→ least privileged mode with limited access to system resources.

② FIQ (fast interrupt request):
→ Handles high-priority, fast interrupts.
→ Provides extra banked registers for every quick response.

③ IRQ mode (interrupt request):
→ Handles normal interrupts
→ User fewer banked registers than FIQ.

④ Supervisor Mode (SVC):
→ Entered on reset or when a software interrupt occurs
→ A fully privileged mode used by operating systems.

⑤ Abort Mode:
→ Entered when a memory access fails
→ Used to handle instruction or data abort exceptions.

⑥ Undefined mode
→ Enters when CPU executes an unknown / unsupported instruction allowing system to handle errors safely

⑦ System Mode (= line)
→ A privileged mode used by the operating system for normal tasks, with same registers as user mode but full access rights.

05½/08½ Pipelining in ARM architecture:

ARM uses instruction pipelining to speed up execution by overlapping different stages of instruction processing.

3-Stage Pipeline.

① Fetch
The processor fetches the instruction from memory

② Decode
The fetched instruction is decoded to understand what operation is required.

③ Execute.
The instruction is executed (ALU, memory access, register update).

How it works:

- which one instruction is in Execute, the next instruction is in Decode, & the next one is in Fetch
- Thus 3 instructions are processed at the same time → faster performance
- Reduces CPU idle time & Improves throughput.

6) Cortex Processors:

→ ARM cortex processor are a family of modern ARM CPU cores designed for high performance, low power & efficient embedded processing.

→ They follow ARMv7 & ARMv8 architectures & are widely used in mobile phones, automotive systems, IOT devices & microcontroller.

① TYPES:
CORTEX-A: for application (os, smartphones).
CORTEX-R: For real-time systems (automotive, robotics)
CORTEX-M: for microcontrollers (IOT, embedded).

② They support Thumb-2 instruction set, giving high performance with smaller code size.

③ Low power consumption.

④ Include in-buit features like memory protection, extensions, etc.

⑤ Used in billions of devices due to their high efficiency, scalability & reliability.

Q7) Features of Cortex ARM Processors

1. Low power consumption - ideal for battery-powered devices
2. High performance - efficient 32-bit processing
3. Thumb-2 instruction set + compact, smaller code size
4. Pipeline Support - supports 3-stage or 5-stage pipeline for faster execution.
5. Faster execution, less waiting time.
6. Low interrupt latency - quick response for real-time applications.
7. Scalable family - Cortex-A, Cortex-R, Cortex M.
8. Highly integrated peripherals - Timers, serial ports, etc.
9. RISC Architecture - Better efficiency
10. Hardware Multiplier & Divider.
   - Arithmetic operations become faster.
11. Built-in debugging support
12. Low Cost
13. Wide ecosystem support - supported by many tools.
14.

19) Cortex-A: (Application Processors)
→ used in smartphones, tablets, laptops, smart TVs.
→ Support high performance, virtual memory OS like Android/Linux.
→ Has MMU & big pipelines.

CORTEX-R: (Real-Time Processors)
→ used in automotive, robotics, hard-disk controllers, safety systems.
→ Designed for real-time performance → very low latency, fast response.
→ Has ECC, TCM & fault-tolerant features.

CORTEX-M: (Microcontroller Processors)
→ Used in IOT devices, sensors, smart home, medical devices, ARM microcontrollers.
→ Very low power, low cost, & simple Thumb-2 instructions.
→ Has NVIC (Nested vectored interrupt Controller) for fast interrupts.

Q10) General features of Cortex ARM processors:

1. They are based on ARM architecture.
2. They are low-power, suitable for battery devices.
3. They use RISC design
4. They support Thumb/Thumb-2 instructions for smaller code size.
5. They have high performance with low power usage.
6. They use pipelining to speed up execution.
7. Widely used in IOT, automation, etc.
8. They are highly scalable
9. They support debugging through SWD
10. They are widely used in smartphones, tablets & ES.
11. They include MMU, MPU, for safety & OS support.
12. They support multiple operating modes for safety & control.
13. Fast processing as they contain/include a large register set.